Robust Aggregation Is Optimization: Anchored Consensus for Byzantine-Robust Knowledge Sharing on the Decentralized Web

Anonymous Author(s)

ABSTRACT

Decentralized Web platforms such as the Fediverse want shared machine-learned services, for example moderation and spam filters, without a central server that sees every instance's data. Knowledge sharing suits them: instances exchange predictions on public content instead of model parameters, so they can run different models, and message size does not depend on model size. But federation is open, so some instances may lie, and every instance must aggregate whatever its neighbours send. We argue that robust aggregation should be treated as what it implicitly is: a decentralized optimization problem. We write knowledge consensus as the minimization of an *anchor*, which keeps each instance near its own evidence, plus a *robust coupling*, which pulls neighbours together with bounded force, and we read every gossip step as one solver step. The resulting family, ARCO, contains gossip averaging, ClippedGossip, BALANCE-style acceptance, geometric-median iterations and the best rule found by an LLM coding agent as special cases. For convex bounded-slope couplings, we prove that the honest optimum moves by at most $\delta\|b\|_2/\kappa$ under arbitrary Byzantine messages, regardless of the solver; we also prove linear convergence into this ball under time-varying attacks. The same view yields a consensus–robustness trade-off in the anchor and predicts a weakness of rules whose scale the attackers can set, which we turn into a new *inlier drift* attack. On 528 pre-registered held-out scenarios, including Usenet news text, and in end-to-end training, robustness forms a three-way frontier between gross-attack robustness, drift robustness and benign accuracy, and no rule dominates. This holds for hand-designed, randomly searched and LLM-designed rules alike. A controlled study with four independent LLM-agent runs finds that feedback improves the design score but not held-out performance.

CCS CONCEPTS

• **Computing methodologies** → **Distributed algorithms**; • **Security and privacy** → *Systems security*; • **Information systems** → *World Wide Web*.

KEYWORDS

decentralized Web, Fediverse, federated learning, Byzantine robustness, robust aggregation, consensus optimization, knowledge distillation, LLM agents

1 INTRODUCTION

The decentralized Web replaces platform-owned servers by independently operated ones that federate. Mastodon, Lemmy and other Fediverse software connect thousands of instances [30, 38], Solid stores personal data in user-controlled pods [27], and Web-of-Things deployments push inference to the edge. These systems increasingly need machine-learned services, such as toxicity and spam filters, content classifiers and recommenders, but they cannot pool data at a central server without giving up the reason they exist. Instances already struggle to moderate alone, and sharing models between them has been proposed as a remedy [3, 9]. The natural way to learn together is decentralized federated learning (DFL): instances train on local data and repeatedly exchange information with the instances they federate with [15, 21, 33].

Federation on the Web is open. Anyone can run an instance, and instances are admitted without vetting, so Sybil and malicious instances are part of the threat model [5, 17]. A learning protocol between instances must therefore aggregate whatever its neighbours send, including messages from instances that lie. This paper studies that aggregation layer, a core component of decentralized and federated Web systems, and also examines an LLM coding agent as the designer of this component. We focus on *knowledge sharing*, in which instances exchange soft predictions on a common set of public content (for example, public posts that are visible across the federation) instead of model parameters, as in distillation-based FL [19, 22]. Knowledge sharing lets instances run different models and keeps a message at $m \times C$ numbers regardless of model size, which matters for volunteer-run servers. Our evaluation includes Usenet news text (20 Newsgroups [18]), which originates from the earliest federated discussion network, alongside two image benchmarks.

Robust aggregation is usually addressed with a growing list of *rules*: coordinate-wise trimming [7], self-centred clipping [10], distance-threshold acceptance [8], geometric medians [29], and their centralized predecessors [4, 13, 37]. Each rule is motivated by the attacks it was designed against, and new attacks typically break a subset of them [1, 32, 36]. More recently, LLM agents have been used to *search* the space of such rules automatically [23, 28, 31]. What is still missing is an account of what these rules *optimize*, and hence of what they can and cannot guarantee.

In this paper, we argue that robust aggregation is best understood as optimization. We formulate decentralized knowledge consensus as the minimization of an explicit objective that consists of an *anchor*, which keeps each node close to its own evidence, and a *robust coupling*, which pulls neighbours together with bounded force. Every gossip step is then one solver step for this objective. We call the resulting family ARCO (*Anchored Robust Consensus Optimization*). This formulation has three consequences. First, it unifies existing rules. Gossip averaging, ClippedGossip, BALANCE, Weiszfeld-type geometric medians and the rule found by an LLM agent are all ARCO instances that differ in four choices: loss, scale, solver and anchor. For example, ClippedGossip is exactly a gradient step on a Huber coupling with zero anchor. Second, it yields guarantees. With a convex bounded-slope coupling and a positive anchor, the honest optimum moves by at most $\delta\|b\|_2/\kappa$ under *any* Byzantine messages, and projected gradient converges linearly into this ball even when attackers change their messages at every iteration. In our measurements the bound is up to 92% tight. The same analysis gives a consensus–robustness trade-off in the anchor strength, and

it explains why rules whose scale is set by the closest received messages give a nearby attacker an amplified share of every update. Third, it shows that the right object to report is a frontier rather than a winner. Building on the amplification result, we construct an *inlier drift* attack that stays inside every clipping radius but always pushes in the same direction. It reveals a three-way frontier between gross-attack robustness, drift robustness and benign accuracy that no rule escapes.

The main contributions of this paper are as follows.

- We formulate robust knowledge consensus as an anchored optimization problem (ARCO) and give a four-axis taxonomy that recovers existing rules as special cases (Sec. 4).
- We prove a bounded-influence theorem that holds for any solver, convergence under arbitrary time-varying Byzantine messages, a consensus–robustness trade-off, and an amplification result that predicts a new attack (Sec. 5).
- We evaluate 17 frozen rules on 528 pre-registered held-out scenarios covering unseen attacks (including a black-box adaptive one), unseen data (including Web text), unseen topologies and up to 40% Byzantine nodes, complemented by end-to-end DFL on CIFAR-10 and 20 Newsgroups (Sec. 6).
- We conduct a controlled study of an LLM coding agent as rule designer, with four independent runs and random-search controls, and show that the best LLM-found rule is an ARCO member that inherits its blind spot (Sec. 7).

Code, all results, the pre-registration, every LLM prompt and response, and an extended version with full tables are available at https://anonymous.4open.science/r/code_for_www27-0ECE/.

2 RELATED WORK

2.1 Machine Learning on the Decentralized Web

Measurement studies document how the Fediverse is centralizing around large instances and the moderation burden of small ones [9, 30, 38]. Bin Zia et al. [3] show that instances can benefit from sharing toxicity models. We study how such sharing can be made robust when some instances are adversarial.

2.2 Byzantine-Robust Aggregation

Centralized rules include Krum [4], coordinate-wise median and trimmed mean [37], the geometric median [29], centred clipping [13] and bucketing [14]. Over graphs, BRIDGE [7] trims coordinates, ClippedGossip [10] clips neighbour differences around the receiver, and BALANCE [8] accepts neighbours within a distance threshold; robust aggregation over graphs is further analyzed in [6, 34]. These works analyze specific rules. We derive rules from an objective, so that their behaviour becomes statements about that objective. Our coupling losses are M-estimators [12], and our MM solver is iteratively reweighted least squares.

2.3 Knowledge-Based FL and LLM-Guided Design

FedMD [19] and FedDF [22] aggregate soft predictions on a public set and distil the result [11]; we replace their server-side average by robust decentralized consensus. FunSearch [31], EoH [23], LLM4AD [24], Eureka [26], DiscoPOP [25] and AlphaEvolve [28] use LLMs to propose programs that are scored and refined. Random search is known to be a strong control in neighbouring areas [2, 20], so we include it, together with independent repeats and held-out tests.

3 PROBLEM SETUP

System. n nodes (instances) communicate over a connected undirected federation graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, and $\mathcal{N}_i$ denotes the neighbours of i. All nodes can read a public, unlabeled set of m items, such as public posts. In each FL round, node i trains its model on private data and computes its *knowledge* $y_i \in \Delta^m$: the $m \times C$ matrix of its softmax outputs on the public set, with each row in the probability simplex Δ. The nodes then run T synchronous consensus iterations. In each iteration, a node sends its current state $x_i \in \Delta^m$ to every neighbour. The final state is the node's distillation target for the next round.

Threat model. A set $\mathcal{B}$ of Byzantine nodes may send arbitrary arrays, possibly different ones to different receivers. They may collude, observe all honest states, and query the honest rule as a black box. $\mathcal{H} = \mathcal{V} \setminus \mathcal{B}$ are the honest nodes. Honest node i has $b_i = |\mathcal{N}_i \cap \mathcal{B}|$ Byzantine neighbours. Nodes know neither $\mathcal{B}$ nor b_i. Some honest nodes also hold heavily mislabeled data, so honest knowledge is itself heterogeneous and partly unreliable; on the Web, an instance whose moderators label carelessly is an example.

Goal. Honest targets should be accurate on the public set and mutually consistent. Ideally they match the *oracle* consensus $\bar{y} = \frac{1}{|\mathcal{H}|}\sum_{i\in\mathcal{H}} y_i$, which would be reached if Byzantine nodes were known and excluded.

4 ANCHORED ROBUST CONSENSUS OPTIMIZATION

Objective. Honest nodes jointly minimize

$$F(X) = \sum_{i\in\mathcal{H}} \frac{\kappa}{2}\|x_i - y_i\|^2 + \frac{1}{2}\sum_{i\in\mathcal{H}}\sum_{j\in\mathcal{N}_i} \rho_\delta\big(\|x_i - x_j\|\big), \quad x_i \in \Delta^m, \tag{1}$$

where $\|\cdot\|$ is the Frobenius norm. For a Byzantine neighbour j, x_j is replaced by whatever message m_j^t it sends. The first term is the **anchor**: node i should not drift far from its own evidence, and $\kappa \geq 0$ sets the anchor's strength relative to the coupling. The second term is the **robust coupling**, where ρ_δ is an M-estimator loss with scale δ and $\rho'_\delta(r) = r\,\omega(r)$, with $\omega(r) = \min(1, \delta/r)$ (Huber), $\min(1, \delta/r)^2$ (log), or 1 (ℓ_2). For both robust losses $|\rho'_\delta| \leq \delta$, so a single neighbour exerts bounded force. The Huber force stays at δ however far the message is. The log loss has force δ^2/r, which redescends towards zero. We apply the loss to the whole message (node-level grouping) rather than to each public item, because being Byzantine is a property of the node.

Solvers. Each consensus iteration performs one step of a solver for (1), using only the messages just received. *Projected gradient:* $x_i \leftarrow \Pi_\Delta\big(x_i + \eta_i[\sum_j \omega(r_{ij})(m_j - x_i) - \kappa(x_i - y_i)]\big)$ with $r_{ij} = \|m_j - x_i\|$, where Π_Δ is the row-wise Euclidean projection onto the simplex. *Proximal majorize–minimize (MM):* because $\rho_\delta(\sqrt{s})$ is concave in s for all three losses, the local objective is majorized by a quadratic with weights $\omega_j = \omega(r_{ij})$. Adding a proximal term

Algorithm 1 ARCO consensus phase at honest node i

Require: local knowledge y_i; loss ω; scale rule (c, q); anchor κ; iterations T

$x_i \leftarrow \Pi_\Delta(y_i)$
for $t = 0, \dots, T-1$ **do**
  send x_i to all $j \in \mathcal{N}_i$; receive $\{m_j\}_{j\in\mathcal{N}_i}$
  $r_j \leftarrow \|m_j - x_i\|$; $\delta \leftarrow c \cdot \text{quantile}_q\{r_j\}$; $\omega_j \leftarrow \omega(r_j;\delta)$
  $x_i \leftarrow \Pi_\Delta\big((x_i + \kappa y_i + \sum_j \omega_j m_j)/(1 + \kappa + \sum_j \omega_j)\big)$ (Eq. 2)
end for
return distillation target x_i

Table 1: Robust gossip rules as instances of ARCO (Eq. 1). None of the existing rules uses an anchor.

Rule	coupling ρ	scale δ	solver	anchor
Gossip averaging	ℓ_2	–	grad/MM	0
ClippedGossip [10]	Huber	quantile	grad	0
BALANCE [8]	trunc. quad.	$\propto \|x_i\|$	damped MM	0
GeoMed (Weiszfeld)	ℓ_1-norm	–	MM (no prox)	0
LLM-found (Sec. 7)	log	quantile	grad (resc.)	0
ARCO	Huber / log	abs./quantile	grad / MM	$\kappa \geq 0$

$\frac{1}{2}\|x - x_i\|^2$ gives the closed-form step

$$x_i \leftarrow \Pi_\Delta\Big(\frac{x_i + \kappa y_i + \sum_{j\in\mathcal{N}_i} \omega_j m_j}{1 + \kappa + \sum_{j\in\mathcal{N}_i} \omega_j}\Big), \tag{2}$$

which decreases the node's local objective for the received messages. *Scale:* δ is either absolute or adaptive, $\delta_i^t = c \cdot \text{quantile}_q\{r_{ij}\}_{j\in\mathcal{N}_i}$. The adaptive scale makes the rule invariant to the overall spread of knowledge, which is large in early rounds and small later. Algorithm 1 shows one consensus phase. Each link carries one $m \times C$ matrix per iteration with no auxiliary state, and each node computes $O(|\mathcal{N}_i|\, mC + mC \log C)$ per iteration.

Existing rules as ARCO instances. Table 1 places common rules on the four axes; the derivations are in Appendix B. Gossip averaging is the ℓ_2 coupling. ClippedGossip [10] is a projected-gradient step on the Huber coupling with $\kappa = 0$ and a quantile scale. BALANCE [8] is a damped MM step on the truncated quadratic ($\omega \in \{0, 1\}$, the limit of redescending losses), with scale proportional to $\|x_i\|$ and a fixed self weight. Weiszfeld's geometric median is MM on $\rho(r) = r$. After ablation, the rule found by our LLM agent is a rescaled gradient step on the log coupling with $\kappa = 0$.

5 THEORY

Let $X = (x_i)_{i\in\mathcal{H}}$ and $\mathcal{X} = (\Delta^m)^{|\mathcal{H}|}$, and let $F_\mathcal{H}$ denote (1) with all Byzantine edges removed, i.e., the problem honest nodes would solve if they knew $\mathcal{B}$. Full proofs are in Appendix A.

Theorem 5.1 (Bounded influence). *Let ρ_δ be convex with $|\rho'_\delta| \leq \delta$ (e.g., Huber), and let $\kappa > 0$. Let $X^\star = \arg\min_\mathcal{X} F_\mathcal{H}$. For arbitrary fixed Byzantine messages $\{m_j\}$, let X^m minimize F over $\mathcal{X}$. Then $\|X^m - X^\star\| \leq \frac{\delta}{\kappa}\,\|b\|_2$, where $b = (b_i)_{i\in\mathcal{H}}$.*

Proof sketch. The Byzantine terms add a gradient whose norm is at most $\delta\|b\|_2$, whatever the messages. Combining the two first-order optimality conditions with the κ-strong convexity of $F_\mathcal{H}$ gives the bound. □

The bound does not depend on what attackers send, on the solver, or on the number of iterations. Both ingredients are necessary. With the ℓ_2 coupling, a single attacker can move X^m arbitrarily far. With $\kappa = 0$, the honest optimum is not unique, because every consensus point minimizes the coupling, so a persistent attacker can choose among them.

Proposition 5.2 (Convergence under time-varying attacks). *Under the assumptions of Theorem 5.1, run projected gradient with step $\eta \leq 1/(\kappa + 2d_{\max})$, where $d_{\max}$ is the maximum degree, against messages m_j^t that may change arbitrarily at every iteration. Then $\|X^t - X^\star\| \leq (1 - \eta\kappa)^t \|X^0 - X^\star\| + \frac{\delta}{\kappa}\|b\|_2$.*

Corollary 5.3 (Consensus–robustness trade-off). *If all honest edges are in the quadratic regime of ρ_δ at $X^\star$, let λ_2 be the algebraic connectivity of the honest graph. Then, for any attack, $\|X^m - \mathbf{1}\bar{y}\| \leq \frac{\kappa}{\kappa+\lambda_2}\|Y - \mathbf{1}\bar{y}\| + \frac{\delta}{\kappa}\|b\|_2$.*

The first term, the honest disagreement left by the anchor, grows with κ. The second, the Byzantine influence, shrinks with κ. Sparse federations (small λ_2) need a weaker anchor to reach consensus and therefore concede more to attackers. No existing rule exposes this dial, and with $\kappa = 0$ the second term is unbounded.

Proposition 5.4 (Inlier amplification). *Consider any update $x_i \leftarrow x_i + \frac{1}{Z}\sum_j \omega_j(m_j - x_i)$ with $Z \geq 1$, which covers both solvers. Let $\omega(r) = \min(1, \delta/r)^p$ and let the scale be self-centred, $\delta = c \cdot \text{quantile}_q\{r_j\}$. Suppose at least $\lceil q|\mathcal{N}_i|\rceil$ Byzantine neighbours send messages at distance $r_B \leq \gamma \min_h r_h$ from x_i, with $\gamma < 1$, where r_h ranges over the h_i honest distances. Then $\delta = c\, r_B$, the weight ratio satisfies $\omega_{\text{Byz}}/\omega_{\text{hon}} \geq (\min(1,c)/\min(1,c\gamma))^p$, and the honest weights satisfy $\sum_h \omega_h \leq h_i \min(1, c\gamma)^p$. Averaging, by contrast, gives every message weight 1.*

Two effects compound. For $c \leq 1$, each attacker outweighs each honest neighbour. More importantly, the total pull towards honest neighbours collapses: for $c = 0.5$, $p = 2$ and $\gamma = 0.9$, each honest neighbour gets weight at most 0.2 instead of 1. Honest nodes then stop contracting towards each other while the attacker pushes in a consistent direction. This motivates the *drift* attack: stay closer than the closest honest neighbour, and always push the same way. A large c or a convex loss removes the effect, but it readmits gross outliers whose distance is comparable to the honest spread. This is the frontier we measure next.

6 EXPERIMENTS

6.1 Protocol

Knowledge, data and graphs. There are 20 nodes on Dirichlet(α) non-IID partitions, and four honest nodes have 60% label noise. Image nodes train small CNNs (conv32–conv64–fc128) on Fashion-MNIST [35] and CIFAR-10 [16]. Text nodes train TF-IDF MLPs (3000–256–20) on 20 Newsgroups [18], which has 20 classes of Usenet posts with headers, footers and quotes removed. For consensus-level studies we use it as unseen *Web text* knowledge. Knowledge is the softmax output on $m = 1000$ public items, recorded at FL rounds 1, 4 and 10 of a clean run. Early knowledge is weak and heterogeneous; late knowledge is strong and similar. Unless stated otherwise, the graph is Erdős–Rényi ($p = 0.3$), 20%

of nodes are Byzantine, and consensus runs $T = 10$ iterations. We report *relative accuracy*: the mean accuracy of honest targets on the public labels, divided by the accuracy of the oracle consensus $\bar{y}$.

Attacks. *Design attacks*, seen during tuning: none, Gaussian, sign flip, label flip (class order reversed), and ALIE [1]. *Held-out attacks*: IPM [36], mimic [14], zero-sum, dissensus [10], targeted (a confident one-hot on a fixed class, the knowledge-space analogue of promoting spam), and *adaptive*. The adaptive attacker queries the frozen rule as a black box at every iteration and for every receiver, and sends the candidate, among 24, that moves the receiver furthest [32]. *Drift* (Sec. 5) sends $x_i + 0.9\, r_{\min} u$, where $r_{\min}$ is the receiver's smallest honest distance and u points towards a fixed class. In *none*, Byzantine nodes report their own knowledge but never update, i.e., they are stubborn rather than malicious.

Pre-registration and selection. All tuning uses only the *design suite*: FMNIST, the five design attacks, rounds $\{1, 4, 10\}$, one Byzantine placement, and the score $\frac{1}{2}$mean+$\frac{1}{2}$worst accuracy. Its *validation* twin uses two fresh placements. Hand-designed baselines get a grid search on this suite. The held-out suites were fixed in a pre-registration file before any automated design run: unseen attacks (36 scenarios), unseen data (CIFAR-10 and FMNIST with $\alpha = 0.1$; 132), unseen topologies (ring, Watts–Strogatz, random geometric, sparse ER; 88), and Byzantine fractions of 10–40% with random or localized placement (176). The drift attack (24 scenarios) and ARCO came after the pre-registration, in response to Sec. 5. The Web-text suite (72 scenarios, all attacks including drift) was added for this version. ARCO configurations were therefore selected on *design-side* data only and frozen before any held-out evaluation: ARCO-G maximizes the validation score (log loss, q=0.5, c=0.5, κ=0.1); ARCO-B maximizes the mean of validation and design-side drift scores (log, q=0.25, c=2, κ=0.1); ARCO-H is the best convex member, the one Theorem 5.1 covers (Huber, q=0.25, c=1, κ=0.1). All three use node-level grouping and the MM solver. Appendix C lists all settings and tuning grids.

6.2 The Robustness Frontier

Fig. 1(a) places 60 rules on the design data: 48 ARCO grid points, 8 tuned hand-designed rules and the 4 selected LLM-designed rules. The Pareto front is formed by ARCO members, ClippedGossip and BALANCE. No LLM-designed program is on it. The rules that reject gross outliers best (small scale and redescending loss: ARCO-G and the LLM-found rules) are the most exposed to drift, as Proposition 5.4 predicts. The rules that resist drift (large scale, or acceptance of most neighbours: ARCO-B, BALANCE) admit gross attacks whose distance is comparable to the honest spread. The same structure appears on held-out data (Fig. 1(b,c)), with benign accuracy as a third axis. BALANCE reaches the knee between gross-attack tail and drift, but pays in the benign case: its distance threshold rejects useful honest neighbours.

6.3 Held-out Generalization

Table 2 reports all frozen rules on all held-out suites. *Tuned hand-designed rules are strong.* After tuning, ClippedGossip and BALANCE beat the coordinate-wise rules, GeoMed and Krum on almost every column. Tuning matters: BALANCE with its default threshold rejected every neighbour on our data (design score 0.64 vs. 0.82 tuned). *The best automated and ARCO rules improve the tail, not the mean.* On mean relative accuracy, the top rules are within 0.01 of each other. They differ in the tail: the minimum over the attacked image scenarios is 0.50 for LLM run A and 0.39 for ARCO-G, versus 0.25 for ClippedGossip and 0.16 for BALANCE. *The adaptive attack is not uniformly the strongest.* Black-box optimization against each frozen rule costs the robust rules 0–8 points relative to no attack. For most rules, targeted messages or label flips are harder; these are valid probability matrices close to honest knowledge (Table 3). In knowledge space, the dangerous messages are the plausible ones.

Web text. 20 Newsgroups knowledge is harder: 20 classes, sparse features and lower agreement between nodes. It reproduces the frontier with lower absolute values (Table 2, right). ARCO-G has the best benign accuracy (0.946) and the best gross-attack minimum (0.726, against 0.28 for ClippedGossip and 0.07 for BALANCE), but one of the lowest drift scores (0.697). ARCO-B has the best drift robustness (0.853) but a gross-attack minimum of only 0.13. LLM run A has the best mean under gross attacks (0.936). That the same three-way trade-off appears on text from a federated discussion network, a modality no rule was tuned on, is the main evidence that the frontier is a property of the aggregation problem and not of the image data.

Byzantine fraction. Up to 20% Byzantine nodes, the stronger rules stay within 6% of the oracle. At 40% with random placement, coordinate-wise trimming and GeoMed degrade sharply (to 0.48–0.58), while BALANCE, ARCO-G and the LLM-found rule hold 0.84–0.88 (Fig. 2). Localized placement, where Byzantine instances cluster as a coordinated group of malicious servers would, is *easier* on average, because most honest nodes have no Byzantine neighbour at all.

6.4 The Anchor and the Bound

Anchor trade-off. Fig. 3 varies only κ in the frozen configurations. This is an analysis and was not used for selection. As Corollary 5.3 predicts, attacked accuracy peaks at an interior κ. For ARCO-B, accuracy under the targeted attack rises from 0.70 at $\kappa = 0$ (no anchor, as in all existing rules) to 0.87 at $\kappa = 0.2$, while disagreement increases monotonically. The anchor also limits accumulation: under drift with up to $T = 160$ iterations, ARCO-G falls from 0.80 to 0.73, while ARCO-B stays within 2.5 points (Fig. 4). Anchor-free rules that accept stubborn neighbours (Mean, BALANCE) lose up to 15 points as T grows.

Tightness of Theorem 5.1. We ran projected-gradient ARCO with an absolute-scale Huber coupling to convergence for 18 settings ($\kappa \in \{0.05, 0.2, 1\}$, $\delta \in \{0.5, 2\}$, three attacks). The bound held in all cases. The measured deviation was 1–9% of the bound for Gaussian noise, 12–65% for sign flip, and 20–92% for drift. The persistent in-radius attacker is the near-worst case the theorem anticipates.

6.5 End-to-end Decentralized Training

Table 4 closes the loop: every node distills from its consensus targets in every round, so aggregation errors compound over 20 rounds of training. Per-attack results with seed spreads are in the extended version.

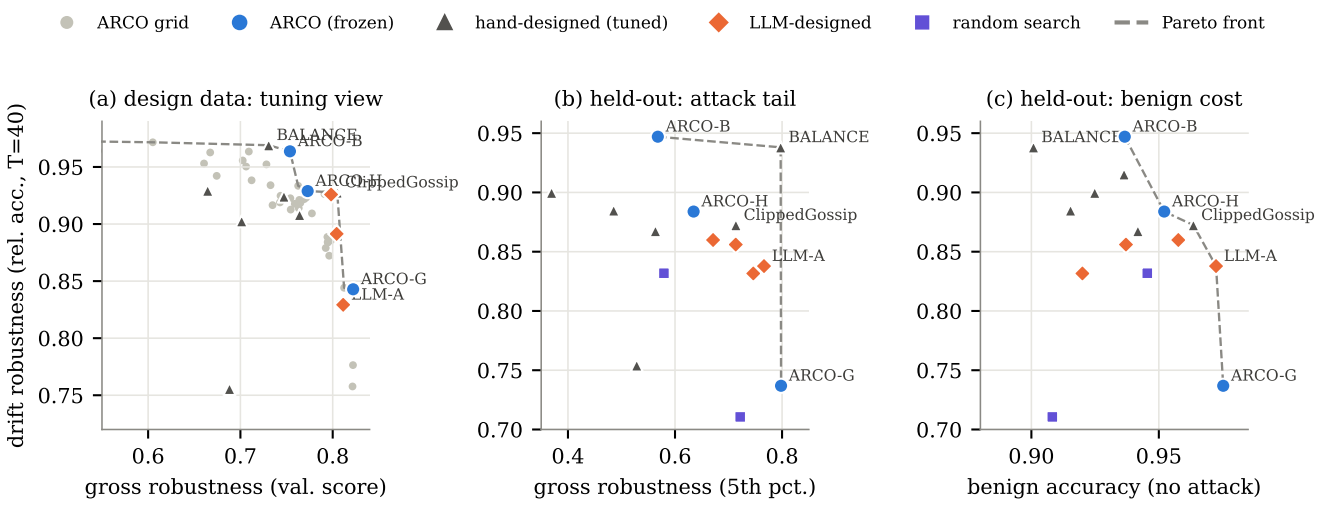


Figure 1: The robustness frontier. (a) Design data, used for tuning and for selecting the frozen ARCO configurations. (b, c) Held-out data. Each point is one rule, and the dashed line is the Pareto front of the panel. Rules trade gross-attack robustness against drift robustness (a, b), and drift robustness against benign accuracy (c).

Table 2: Held-out evaluation of frozen rules (relative accuracy = honest consensus accuracy / oracle accuracy). Image suites: 432 scenarios (396 attacked) plus 24 drift scenarios. Web text: 20 Newsgroups knowledge, 72 scenarios, all 12 attacks. Design val. is the only number any rule was tuned or selected on. Bold: best in column. No rule is best in all columns.

	design	image: mean over held-out suite				image: gross tail		image	image: drift		Web text (20 Newsgroups)			
Rule	val.	attacks	data	topology	Byz. frac.	5th pct.	min	benign	T=10	T=40	attacked	min	benign	drift
Hand-designed (tuned on design suite)														
Mean	0.191	0.593	0.530	0.567	0.566	0.122	0.011	0.832	0.942	0.936	0.425	0.048	0.740	0.764
CW-Median	0.747	0.936	0.890	0.823	0.820	0.485	0.166	0.915	0.890	0.885	0.735	0.120	0.799	0.748
BRIDGE-T	0.701	0.904	0.923	0.829	0.796	0.369	0.163	0.925	0.905	0.900	0.781	0.183	0.841	0.772
Krum	0.688	0.872	0.815	0.681	0.731	0.528	0.180	0.806	0.756	0.754	0.715	0.244	0.728	0.613
GeoMed	0.764	0.954	0.943	0.843	0.845	0.563	0.179	0.942	0.875	0.867	0.857	0.210	0.885	0.746
Huber (IRLS)	0.665	0.913	0.920	0.828	0.814	0.303	0.110	0.936	0.915	0.915	0.851	0.084	0.862	0.793
ClippedGossip	0.805	0.971	0.948	0.882	0.907	0.714	0.247	0.964	0.896	0.872	0.899	0.283	0.914	0.793
BALANCE	0.731	0.971	0.967	**0.909**	**0.940**	0.797	0.158	0.901	0.943	0.938	0.903	0.067	0.833	0.821
Automated search														
Random search (24)	0.775	0.951	0.924	0.841	0.879	0.722	0.383	0.908	0.778	0.711	0.818	0.439	0.839	0.673
Random search (200)	0.788	0.932	0.918	0.856	0.876	0.579	0.299	0.945	0.875	0.832	0.852	0.156	0.902	0.787
LLM run A	0.811	0.975	**0.969**	0.904	**0.940**	0.766	**0.500**	0.972	0.839	0.838	**0.936**	0.567	0.933	0.725
LLM run B	0.799	**0.979**	0.961	0.881	0.907	0.746	0.179	0.920	0.841	0.832	0.880	0.067	0.848	0.677
LLM run C	0.798	0.953	0.920	0.864	0.895	0.713	0.308	0.937	0.867	0.856	0.846	0.274	0.872	0.751
LLM run H (small)	0.804	0.957	0.962	0.872	0.879	0.671	0.180	0.958	0.872	0.860	0.907	0.624	0.942	0.757
ARCO (ours, frozen)														
ARCO-G	**0.822**	0.977	0.966	0.893	0.935	**0.798**	0.391	**0.975**	0.770	0.737	0.917	**0.726**	**0.946**	0.697
ARCO-B	0.754	0.950	0.924	0.865	0.880	0.568	0.151	0.937	**0.954**	**0.947**	0.894	0.129	0.899	**0.853**
ARCO-H	0.773	0.960	0.947	0.868	0.889	0.635	0.179	0.952	0.905	0.884	0.897	0.140	0.899	0.791

Collaboration survives the attack, averaging does not. Under the six non-drift attacks, every robust rule beats local-only training on attacked mean, by 3.0–10.1 points on CIFAR-10 and 4.3–13.4 points on 20 Newsgroups. Gossip averaging falls to a worst case of 18.4% and 8.8%, far below local training. On 20 Newsgroups, parameter averaging (DFedAvg) has the best benign accuracy (40.7%), but it requires identical architectures and sends about four times more floats per link than knowledge sharing.

Table 3: Per-attack relative accuracy on unseen datasets (CIFAR-10, FMNIST-0.1; suite `heldout_data`). †: attack never seen during design. Adaptive = black-box optimisation against the frozen rule.

Attack	Mean	BRIDGE	GeoMed	ClipG.	BALANCE	LLM run A	ARCO-G	ARCO-B
none	0.86	0.97	0.98	1.00	0.94	**1.01**	1.00	0.97
gaussian	0.28	0.95	**1.00**	0.98	1.00	0.99	0.97	**1.00**
signflip	0.16	0.94	0.91	0.90	**0.99**	0.99	0.96	0.92
labelflip	0.09	0.87	0.91	0.91	0.84	**0.97**	0.96	0.77
alie	**1.00**	0.97	**1.00**	0.98	**1.00**	0.99	0.97	**1.00**
ipm†	0.99	0.95	**0.99**	0.98	0.99	0.99	0.97	**1.00**
mimic†	**0.96**	0.87	0.88	**0.96**	0.96	0.94	0.95	**0.96**
zerosum†	0.35	0.93	0.99	0.98	**1.00**	0.99	0.97	**1.00**
dissensus†	0.53	0.92	0.94	0.91	**0.96**	0.89	**0.96**	0.91
targeted†	0.15	0.84	0.82	0.86	**0.98**	0.94	0.96	0.75
adaptive†	0.45	0.95	0.97	0.97	**0.99**	0.97	0.95	0.88

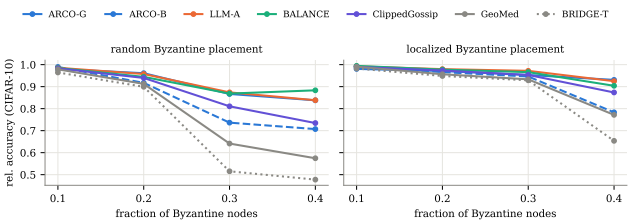


Figure 2: Relative accuracy on CIFAR-10 knowledge vs. fraction of Byzantine nodes (mean over 11 attacks and 2 placement seeds).

Table 4: End-to-end DFL (20 nodes, 20 rounds, 20% Byzantine): honest test accuracy (%). *mean*/*worst*: over the six non-drift attacks (sign flip, ALIE, IPM, dissensus, targeted, adaptive). CIFAR-10: one or two seeds per cell; 20 Newsgroups: three seeds. DFedAvg exchanges parameters and cannot express several knowledge-space attacks (–). Per-attack results are in the extended version.

	CIFAR-10				20 Newsgroups (Web text)			
Method	benign	mean	worst	drift	benign	mean	worst	drift
Local only	32.4	32.4	32.4	32.4	21.7	21.7	21.7	21.7
DFedAvg (param.)	40.8	–	–	–	40.7	–	–	–
Gossip averaging	40.2	33.4	18.4	41.6	31.4	21.8	8.8	32.5
CW-Median	**44.0**	41.9	36.5	**41.8**	37.2	31.8	26.6	29.7
BRIDGE-T	43.8	41.7	34.6	41.6	**37.5**	33.1	27.8	31.1
GeoMed	42.0	39.9	34.6	39.3	35.3	32.1	24.1	30.5
ClippedGossip	42.2	41.0	36.7	39.8	35.3	32.3	24.3	31.4
BALANCE	40.7	42.4	37.5	41.5	31.8	**35.1**	32.7	32.8
Random search (200)	40.9	39.5	30.0	–	34.7	28.5	17.3	29.2
LLM run H	43.3	**42.5**	**40.1**	–	35.8	34.5	**33.7**	27.0
LLM run A	42.5	37.9	33.9	35.7	36.1	29.4	21.2	28.1
ARCO-G	42.8	39.7	37.5	34.5	35.6	31.2	27.8	26.8
ARCO-B	41.3	35.4	33.0	41.5	34.5	27.8	24.4	**33.1**
ARCO-H	41.8	35.7	29.9	40.1	34.7	26.0	18.0	31.8

The drift side of the frontier carries over to training. The rules that reject gross outliers best at the consensus level, ARCO-G and LLM run A, are among the weakest under drift on both datasets: ARCO-G has the lowest drift accuracy of all rules on both (34.5% and 26.8%). ARCO-B, selected for drift robustness on design data, is the best rule under drift on 20 Newsgroups (33.1%) and within 0.3 points of the best on CIFAR-10. So Proposition 5.4 predicts an end-to-end weakness, not only a consensus-level one.

The gross-attack side does not carry over. At the consensus level, ARCO-G had the best gross-attack minimum on Web-text knowledge. End-to-end, the best attacked means come from BALANCE (42.4% and 35.1%) and LLM run H (42.5% and 34.5%), and ARCO-G is in the middle of the pack (39.7% and 31.2%). ARCO-B and ARCO-H are weaker still under sign flip and IPM, and ARCO-H falls below local training under sign flip on 20 Newsgroups (18.0%). The training curves show an early plateau rather than slow accumulation: under these attacks, ARCO stops improving after about five rounds. The cause is not target accuracy alone. On 20 Newsgroups, averaged over the non-drift attacks, ARCO-G's targets classify the public items as accurately as BALANCE's (54.7% vs. 53.7%), yet its students are 4 points worse. What a target transmits beyond its argmax therefore matters for distillation, and our consensus-level metric does not capture it. We report this as a limitation of consensus-level evaluation: it predicts the drift weakness but overpredicts gross-attack robustness in training.

6.6 Deployment Cost

Communication. A knowledge message is an $m \times C$ matrix, independent of the model, so each consensus iteration sends mC floats per link. With $T = 10$ iterations per round, 20 rounds of knowledge sharing send 2.0×10^6 floats per link on CIFAR-10 and 4.0×10^6 on 20 Newsgroups, whereas DFedAvg sends 1.09×10^7 and 1.55×10^7 for our small models, i.e., 5.5 and 3.9 times more. The gap grows linearly with model size, and it can be traded against robustness through T: Theorem 5.1 bounds the fixed point, and even under drift the anchored configurations lose at most 7 points between $T = 5$ and $T = 160$ (Fig. 4). Unlike population-based search, every ARCO step sends a single message per neighbour.

Computation. An MM step costs one pass over the received messages plus a projection onto the simplex, i.e., $O(|\mathcal{N}_i|\, mC)$ per node. In our simulator, which runs the consensus of all 20 nodes sequentially on one laptop CPU, one round of consensus takes 15 ms for averaging, 34–36 ms for ClippedGossip and BALANCE, 90–93 ms for ARCO and 0.55 s for GeoMed on CIFAR-10 knowledge; on 20 Newsgroups the corresponding times are 42 ms, 72–80 ms, 0.20–0.23 s and 1.03 s. Per instance, this is negligible next to local training.

Public content. Knowledge sharing needs a set of items that every instance can score. On the Fediverse, public posts and hashtags are already replicated across instances, so the reference set requires no additional data release; instances only reveal their predictions on content that is already public. Whether predictions on public items leak information about an instance's private training data is a separate privacy question that we do not address.

7 CAN AN LLM AGENT ESCAPE THE FRONTIER?

LLM coding agents are increasingly proposed as designers of systems components. We asked whether one finds an aggregation rule outside the family.

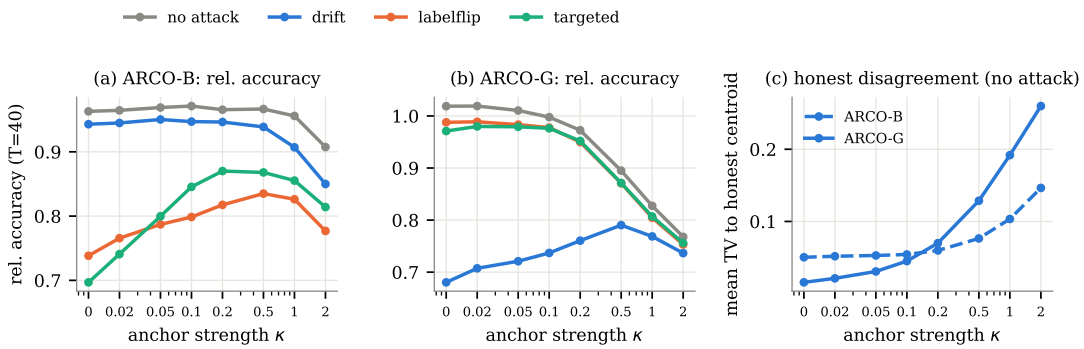


Figure 3: Anchor strength κ ($T = 40$, held-out data). Attacked accuracy peaks at an interior value (a, b) while honest disagreement grows monotonically (c), as Corollary 5.3 predicts. $\kappa = 0$ corresponds to the existing anchor-free rules.

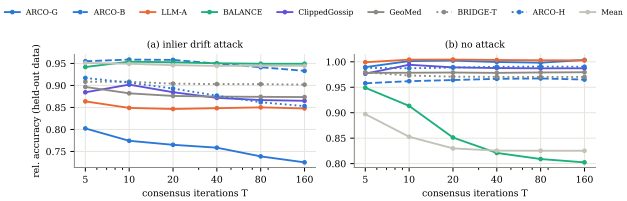


Figure 4: Effect of the number of consensus iterations T under drift (a) and with stubborn but non-malicious Byzantine nodes (b).

7.1 Setup

We use a generate–evaluate–refine loop [23, 31]. In each of 6 rounds the agent receives the task, the interface, four tuned baselines with their design-suite scores, and, from round 2 on, its five best previous programs with per-attack feedback. It returns four new programs. The agent sees only the prompt: no code base and no held-out scenarios. We ran three independent runs with a large model (Claude Opus 5.5; runs A, B, C) and one with a small model (Claude Haiku 5.5; run H). This gave 96 programs from 24 LLM calls, all of which parsed and ran. Selection followed the pre-registered rule (best validation score). The controls are random search over a hand-written grammar of 4 centres × 8 dissimilarities × 4 weightings × memory, self weight and projection, run with the same budget (24 samples) and with 200 samples.

7.2 Findings

(1) *The agent beats random search on design data, but not clearly on held-out data.* Every run exceeds the median random-search run within four programs (Fig. 5). On held-out data, run A is among the strongest rules in Table 2, but runs B, C and H are not clearly better than tuned ClippedGossip. (2) *Feedback rounds optimize the feedback suite.* The best design score keeps rising over the rounds, but held-out-lite accuracy per round stays flat (run A: 0.590–0.611; run C: 0.580–0.609). In runs A and C the selected program comes from round 1. Programs chosen by *design* score instead have worse held-out minima (0.18 vs. 0.50 for run A). Design score is a weak predictor of held-out accuracy within a run ($\rho = 0.05$–0.64, Table 5).

Table 5: LLM-guided design runs (6 rounds × 4 programs each). Sel.: program chosen by the pre-registered rule (best validation score) and the round it was proposed in. Held-out columns as in Table 2 (gross = all non-drift held-out scenarios). ρ: Spearman correlation between design score and held-out-lite accuracy over the run's 24 programs (random search: 0.74 over 200).

Run	LLM	sel. (round)	design	val.	gross mean	gross min	benign	drift	ρ
A	Opus	A1.4 (1)	0.836	0.811	0.942	0.500	0.972	0.838	0.46
B	Opus	B2.3 (2)	0.819	0.799	0.925	0.179	0.920	0.832	0.64
C	Opus	C1.3 (1)	0.803	0.798	0.898	0.308	0.937	0.856	0.05
H	Haiku	H5.1 (5)	0.841	0.804	0.905	0.180	0.958	0.860	0.43

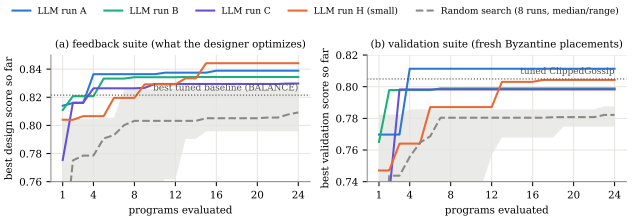


Figure 5: LLM-agent search vs. random search with the same budget. (a) Best design score so far. (b) Best validation score so far. Feedback improves what the agent optimizes more than what generalizes.

(3) *Run-to-run variance is as large as the gap between methods.* The four selected programs span 0.18–0.50 in held-out minimum, so a single agent run is not a reproducible result. (4) *The best program is an ARCO member, and the agent's explanation of it is wrong.* The agent described run A's rule as "self-centred clipped gossip on simplex-projected messages, plus a per-neighbour reputation". Removing both mechanisms changes nothing (Table 6). What matters is that the clipping weight is applied twice, so the influence δ^2/r redescends, which is the log coupling of Sec. 4. The scale constant also matters. Changing either moves the rule along the frontier: Huber weights raise drift robustness from 0.84 to 0.89 but lower the minimum from 0.50 to 0.15. The agent found a good point of the

Table 6: Ablation of the best LLM-found rule. The two mechanisms the LLM described as its key ideas (sanitisation, reputation) do not matter; the loss exponent p and the scale c do, and they move the rule along the frontier rather than improving it.

Variant	gross mean	5th pct.	min	benign	drift
A1.4, full LLM-found rule	0.942	0.766	0.500	0.972	0.838
− input sanitisation	0.943	0.774	0.449	0.972	0.845
− reputation memory	0.940	0.755	0.501	0.977	0.841
− both (= core)	0.941	0.777	0.446	0.977	0.850
core, step $1/(k{+}1)$	0.943	0.786	0.483	0.976	0.851
core, no output projection	0.939	0.777	0.607	0.977	0.849
core, scale $c = 0.5$	0.943	0.824	0.696	0.976	0.662
core, Huber weights ($p = 1$)	0.892	0.571	0.151	0.945	0.893
ClippedGossip + sanit.	0.928	0.726	0.247	0.964	0.872
ClippedGossip (tuned)	0.916	0.714	0.247	0.964	0.872
A4.4 (best *design* score)	0.933	0.703	0.181	0.956	0.870

family at its gross-robust end, because its feedback contained only gross attacks. For agent-designed Web infrastructure, the lesson is that the agent's score and its account of why a design works must both be checked independently.

8 DISCUSSION AND LIMITATIONS

8.1 Implications for Federated Web Systems

It tells an operator which property comes from which choice. Bounded per-neighbour influence comes from a bounded-slope loss, and a bounded fixed point comes from the anchor. A redescending loss rejects far outliers, while a scale that attackers can set exposes the node to near attackers. It replaces the question "which rule?" with "which point on the frontier, given my threat model?". A federation worried about crashed or misconfigured instances sending garbage should sit at the gross-robust end (ARCO-G). One worried about coordinated instances that mimic honest knowledge and persist, such as a spam campaign run from several servers, should sit at the drift-robust end (ARCO-B). In end-to-end training, however, the ARCO configurations do not beat simple median-type rules or BALANCE under most attacks (Sec. 6.5). At present, therefore, ARCO is more an analysis tool than a recommended rule.

Open federation also lets an attacker create many instances cheaply [5]. Our experiments show that the robust rules hold up to 40% Byzantine nodes with random placement, but a Sybil attacker that federates many instances with a single victim controls that victim's neighbourhood, and no local rule can recover from a Byzantine majority among a node's neighbours. The anchor limits the damage in this case: by Theorem 5.1, the deviation of the honest optimum grows with the number of Byzantine neighbours only through $\|b\|_2$ and is always divided by κ, so a stronger anchor caps it at the cost of consensus. Admission control at the federation level, such as the shared blocklists that Fediverse instances already maintain, remains necessary and is complementary to robust aggregation.

8.2 Consensus-Level vs. End-to-End Evaluation

Studies of robust aggregation usually evaluate either the aggregated output or the trained model. We evaluated both, and the two levels agree on drift but not on gross attacks (Sec. 6.5). Two properties of training, which a consensus snapshot does not have, are plausible reasons; we have not yet tested either. First, in training, the knowledge a node contributes is produced by a model that was distilled on earlier targets, so a rule's errors feed back into its own inputs, while a snapshot freezes this loop. Second, distillation uses the full probability vector, whereas relative accuracy uses only the argmax. On 20 Newsgroups, ARCO-G's targets match BALANCE's argmax accuracy on the public items but produce weaker students. We therefore recommend reporting both levels. A consensus-level metric that predicts end-to-end accuracy, for example one based on the calibration of targets, is an open problem.

8.3 Future Directions

Because ARCO makes the frontier explicit in a few parameters (loss, q, c, κ), an instance could move along it at run time, for example by enlarging its scale when it observes persistent in-radius disagreement in one direction. Any such adaptation is itself attackable and must be evaluated against attackers that know it. Real federations are asynchronous: instances join, leave and run at different rates. Our convergence result allows time-varying Byzantine messages but assumes a fixed honest graph and synchronous rounds. Finally, our LLM study shows that agents overfit their feedback suite. Rotating the suite, or including drift-type attacks in it, is a direct test of whether an agent can find designs off the frontier rather than a different point on it.

8.4 Limitations

(i) Scale: we use 20 nodes, small models, and 10- and 20-class tasks; we have no deployment on a live federation. (ii) The end-to-end runs use one to three seeds per cell. The gap between ARCO and the median-type rules reproduces across seeds, but we have not established its cause. (iii) Theorem 5.1 covers convex couplings with a fixed scale. ARCO-G and ARCO-B use a redescending loss and an adaptive scale, for which we have the amplification analysis but no fixed-point bound. (iv) The drift attack, ARCO and the Web-text suite were developed after the pre-registration. We mitigated this by selecting on design-side data only, and we disclose that earlier exploratory screens touched held-out data. (v) We assume a public reference set and synchronous rounds. (vi) The LLM study uses four runs of two models from one provider.

9 CONCLUSION

Robust aggregation for knowledge sharing between Web instances can be written, analyzed and tuned as an optimization problem: an anchor plus a bounded-force coupling. Doing so explains why existing rules work, where their guarantees end, and which attack they are blind to. It also explains why neither hand design nor an LLM agent produces a rule that is best everywhere. The frontier is the object to report, and we hope future proposals, whether human- or agent-designed, will be placed on it. To make this easy, we release ARCO, the frozen configurations and every evaluation suite used in this paper.

REFERENCES

[1] Gilad Baruch, Moran Baruch, and Yoav Goldberg. 2019. A Little Is Enough: Circumventing Defenses For Distributed Learning. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/ec1c59141046cd1866bbbcdfb6ae31d4-Paper.pdf

[2] James Bergstra and Yoshua Bengio. 2012. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research* 13, 10 (2012), 281–305. http://jmlr.org/papers/v13/bergstra12a.html

[3] Haris Bin Zia, Aravindh Raman, Ignacio Castro, Ishaku Hassan Anaobi, Emiliano De Cristofaro, Nishanth Sastry, and Gareth Tyson. 2022. Toxicity in the Decentralized Web and the Potential for Model Sharing. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 2 (May 2022), 1–25. https://doi.org/10.1145/3530901

[4] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. 2017. Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/f4b9ec30ad9f68f89b29639786cb62ef-Paper.pdf

[5] John R. Douceur. 2002. *The Sybil Attack*. Springer Berlin Heidelberg, 251–260. https://doi.org/10.1007/3-540-45748-8_24

[6] El Mahdi El-Mhamdi, Sadegh Farhadkhani, Rachid Guerraoui, Arsany Guirguis, Lê-Nguyên Hoang, and Sébastien Rouault. 2021. Collaborative Learning in the Jungle (Decentralized, Byzantine, Heterogeneous, Asynchronous and Nonconvex Learning). In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 25044–25057. https://proceedings.neurips.cc/paper_files/paper/2021/file/d2cd33e9c0236a8c2d8bd3fa91ad3acf-Paper.pdf

[7] Cheng Fang, Zhixiong Yang, and Waheed U. Bajwa. 2022. BRIDGE: Byzantine-Resilient Decentralized Gradient Descent. *IEEE Transactions on Signal and Information Processing over Networks* 8 (2022), 610–626. https://doi.org/10.1109/tsipn.2022.3188456

[8] Minghong Fang, Zifan Zhang, Hairi, Prashant Khanduri, Jia Liu, Songtao Lu, Yuchen Liu, and Neil Gong. 2024. Byzantine-Robust Decentralized Federated Learning. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, 2874–2888. https://doi.org/10.1145/3658644.3670307

[9] Anaobi Ishaku Hassan, Aravindh Raman, Ignacio Castro, Haris Bin Zia, Emiliano De Cristofaro, Nishanth Sastry, and Gareth Tyson. 2021. Exploring content moderation in the decentralised web: the pleroma case. In *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '21)*. ACM, 328–335. https://doi.org/10.1145/3485983.3494838

[10] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. 2023. Byzantine-Robust Decentralized Learning via ClippedGossip. arXiv:2202.01545 [cs.LG] https://arxiv.org/abs/2202.01545

[11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531 [stat.ML] https://arxiv.org/abs/1503.02531

[12] Peter J. Huber. 1964. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics* 35, 1 (March 1964), 73–101. https://doi.org/10.1214/aoms/1177703732

[13] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. 2021. Learning from History for Byzantine Robust Optimization. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 5311–5319. https://proceedings.mlr.press/v139/karimireddy21a.html

[14] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. 2023. Byzantine-Robust Learning on Heterogeneous Datasets via Bucketing. arXiv:2006.09365 [cs.LG] https://arxiv.org/abs/2006.09365

[15] Anastasia Koloskova, Nicolas Loizou, Sadra Boreiri, Martin Jaggi, and Sebastian Stich. 2020. A Unified Theory of Decentralized SGD with Changing Topology and Local Updates. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 5381–5393. https://proceedings.mlr.press/v119/koloskova20a.html

[16] A. Krizhevsky. 2009. Learning Multiple Layers of Features from Tiny Images.

[17] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems* 4, 3 (1982), 382–401. https://doi.org/10.1145/357172.357176

[18] Ken Lang. 1995. *NewsWeeder: Learning to Filter Netnews*. Elsevier, 331–339. https://doi.org/10.1016/b978-1-55860-377-6.50048-7

[19] Daliang Li and Junpu Wang. 2019. FedMD: Heterogenous Federated Learning via Model Distillation. arXiv:1910.03581 [cs.LG] https://arxiv.org/abs/1910.03581

[20] Liam Li and Ameet Talwalkar. 2020. Random Search and Reproducibility for Neural Architecture Search. In *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference (Proceedings of Machine Learning Research, Vol. 115)*, Ryan P. Adams and Vibhav Gogate (Eds.). PMLR, 367–377. https://proceedings.mlr.press/v115/li20c.html

[21] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. 2017. Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/f75526659f31040afeb61cb7133e4e6d-Paper.pdf

[22] Tao Lin, Lingjing Kong, Sebastian U Stich, and Martin Jaggi. 2020. Ensemble Distillation for Robust Model Fusion in Federated Learning. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 2351–2363. https://proceedings.neurips.cc/paper_files/paper/2020/file/18df51b97ccd68128e994804f3eccc87-Paper.pdf

[23] Fei Liu, Tong Xialiang, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 32201–32223. https://proceedings.mlr.press/v235/liu24bs.html

[24] Fei Liu, Rui Zhang, Zhuoliang Xie, Rui Sun, Kai Li, Qinglong Hu, Ping Guo, Xi Lin, Xialiang Tong, Mingxuan Yuan, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2026. LLM4AD: A Platform for Algorithm Design with Large Language Model. arXiv:2412.17287 [cs.AI] https://arxiv.org/abs/2412.17287

[25] Chris Lu, Samuel Holt, Claudio Fanconi, Alex J. Chan, Jakob Foerster, Mihaela van der Schaar, and Robert Tjarko Lange. 2024. Discovering Preference Optimization Algorithms with and for Large Language Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 86528–86573. https://doi.org/10.52202/079017-2748

[26] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-Level Reward Design via Coding Large Language Models. arXiv:2310.12931 [cs.RO] https://arxiv.org/abs/2310.12931

[27] Essam Mansour, Andrei Vlad Sambra, Sandro Hawke, Maged Zereba, Sarven Capadisli, Abdurrahman Ghanem, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. A Demonstration of the Solid Platform for Social Web Applications. In *Proceedings of the 25th International Conference Companion on World Wide Web - WWW '16 Companion (WWW '16 Companion)*. ACM Press, 223–226. https://doi.org/10.1145/2872518.2890529

[28] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs.AI] https://arxiv.org/abs/2506.13131

[29] Krishna Pillutla, Sham M. Kakade, and Zaid Harchaoui. 2022. Robust Aggregation for Federated Learning. *IEEE Transactions on Signal Processing* 70 (2022), 1142–1154. https://doi.org/10.1109/tsp.2022.3153135

[30] Aravindh Raman, Sagar Joglekar, Emiliano De Cristofaro, Nishanth Sastry, and Gareth Tyson. 2019. Challenges in the Decentralised Web: The Mastodon Case. In *Proceedings of the Internet Measurement Conference (IMC '19)*. ACM, 217–229. https://doi.org/10.1145/3355369.3355572

[31] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2023. Mathematical discoveries from program search with large language models. *Nature* 625, 7995 (Dec. 2023), 468–475. https://doi.org/10.1038/s41586-023-06924-6

[32] Virat Shejwalkar and Amir Houmansadr. 2021. Manipulating the Byzantine: Optimizing Model Poisoning Attacks and Defenses for Federated Learning. In *Proceedings 2021 Network and Distributed System Security Symposium (NDSS 2021)*. Internet Society. https://doi.org/10.14722/ndss.2021.24498

[33] Tao Sun, Dongsheng Li, and Bao Wang. 2023. Decentralized Federated Averaging. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 4 (April 2023), 4289–4301. https://doi.org/10.1109/tpami.2022.3196503

[34] Zhaoxian Wu, Tianyi Chen, and Qing Ling. 2023. Byzantine-Resilient Decentralized Stochastic Optimization With Robust Aggregation Rules. *IEEE Transactions on Signal Processing* 71 (2023), 3179–3195. https://doi.org/10.1109/tsp.2023.3300629

[35] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747 [cs.LG] https://arxiv.org/abs/1708.07747

[36] Cong Xie, Oluwasanmi Koyejo, and Indranil Gupta. 2020. Fall of Empires: Breaking Byzantine-tolerant SGD by Inner Product Manipulation. In *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference (Proceedings of Machine*

Learning Research, Vol. 115), Ryan P. Adams and Vibhav Gogate (Eds.). PMLR, 261–270. https://proceedings.mlr.press/v115/xie20a.html

[37] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. 2018. Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 5650–5659. https://proceedings.mlr.press/v80/yin18a.html

[38] Matteo Zignani, Sabrina Gaito, and Gian Paolo Rossi. 2018. Follow the "Mastodon": Structure and Evolution of a Decentralized Online Social Network. *Proceedings of the International AAAI Conference on Web and Social Media* 12, 1 (2018), 541–550. https://doi.org/10.1609/icwsm.v12i1.14988

A PROOFS

Theorem 5.1. Write $F = F_{\mathcal{H}} + \Phi$ with $\Phi(X) = \sum_i \sum_{j \in \mathcal{N}_i \cap \mathcal{B}} \rho_\delta(\|x_i - m_j\|)$. The gradient g of Φ has blocks $g_i = \sum_j \rho'_\delta(r_{ij}) \frac{x_i - m_j}{r_{ij}}$, so $\|g_i\| \le \delta b_i$ and $\|g\| \le \delta \|b\|_2$ for *any* messages. First-order optimality over the convex set $\mathcal{X}$ gives $\langle \nabla F_{\mathcal{H}}(X^m) + g(X^m), X^\star - X^m \rangle \ge 0$ and $\langle \nabla F_{\mathcal{H}}(X^\star), X^m - X^\star \rangle \ge 0$. Add the two inequalities. Since $F_{\mathcal{H}}$ is κ-strongly convex (the anchor is κ-strongly convex and the coupling is convex), $\kappa \|X^m - X^\star\|^2 \le \langle \nabla F_{\mathcal{H}}(X^m) - \nabla F_{\mathcal{H}}(X^\star), X^m - X^\star \rangle \le \langle g, X^\star - X^m \rangle \le \delta \|b\|_2 \|X^m - X^\star\|$. □

Proposition 5.2. $\nabla F_{\mathcal{H}}$ is L-Lipschitz with $L \le \kappa + 2d_{\max}$: the Huber Hessian is at most the identity per edge, and the honest graph Laplacian has spectral norm at most $2d_{\max}$. The iteration is $X^{t+1} = \Pi_{\mathcal{X}}(X^t - \eta \nabla F_{\mathcal{H}}(X^t) - \eta g^t)$ with $\|g^t\| \le \delta \|b\|_2$, and $X^\star = \Pi_{\mathcal{X}}(X^\star - \eta \nabla F_{\mathcal{H}}(X^\star))$. By non-expansiveness of $\Pi_{\mathcal{X}}$ and the contraction of a gradient step on a κ-strongly convex, L-smooth function for $\eta \le 1/L$, $\|X^{t+1} - X^\star\| \le (1 - \eta\kappa)\|X^t - X^\star\| + \eta\delta\|b\|_2$. Unrolling and summing the geometric series gives the claim. □

Corollary 5.3. In the quadratic regime the optimality condition is linear, $\kappa(X - Y) + L_{\mathcal{H}} X = 0$, so $X^\star = \kappa(\kappa I + L_{\mathcal{H}})^{-1} Y$. The matrix $\kappa(\kappa I + L_{\mathcal{H}})^{-1}$ is row-stochastic and entrywise non-negative, so the solution stays in $\mathcal{X}$ and the simplex constraint is inactive. It preserves the mean $\bar{y}$ and shrinks every component orthogonal to $\mathbf{1}$ by at least $\kappa/(\kappa + \lambda_2)$. Adding Theorem 5.1 through the triangle inequality completes the proof. □

Proposition 5.4. The b_i smallest distances all equal r_B and make up at least a q-fraction of the neighbourhood, so the q-quantile is r_B and $\delta = c\, r_B$. Hence $\omega_{\text{Byz}} = \min(1, c)^p$ and $\omega_h = \min(1, c\, r_B / r_h)^p \le \min(1, c\gamma)^p$. □

B EXISTING RULES AS ARCO INSTANCES

Let $D_j = m_j - x_i$ and $r_j = \|D_j\|$, and let $k = |\mathcal{N}_i|$. *Gossip averaging* (ℓ_2, $\omega \equiv 1$, $\kappa = 0$): the MM step (2) is $(x_i + \sum_j m_j)/(1 + k)$. *ClippedGossip* [10] updates $x_i + \frac{1}{k+1} \sum_j \text{clip}_\tau(D_j)$ with $\text{clip}_\tau(D) = D \min(1, \tau/\|D\|)$. This equals $x_i - \eta \nabla_{x_i} \sum_j \rho_\tau^{\text{Hub}}(\|x_i - m_j\|)$ with $\eta = 1/(k+1)$, i.e., a gradient step on the Huber coupling with $\kappa = 0$. Our tuned radius is $\tau = 0.5 \cdot \text{quantile}_{0.25}\{r_j\}$. *BALANCE* [8] accepts j if $r_j \le \gamma e^{-\lambda t/T} \|x_i\|$ and sets $x_i \leftarrow \alpha x_i + (1-\alpha)\,\text{mean}_{\text{acc}}(m_j)$. With the truncated quadratic $\rho(r) = \frac{1}{2}\min(r^2, \delta^2)$, $\omega_j = \mathbb{1}[r_j \le \delta]$, the minimizer of the majorizer is the mean of accepted messages. BALANCE is therefore a damped MM step with fixed self weight α and scale $\delta = \gamma e^{-\lambda t/T} \|x_i\|$. *Weiszfeld* (GeoMed): $\rho(r) = r$, $\omega(r) = 1/r$, and MM without the proximal term gives $\sum_j (m_j/r_j) / \sum_j (1/r_j)$. *LLM-found rule*: after the ablation in Table 6, the update is $x_i + \frac{1}{k+1} \sum_j \min(1, \tau/r_j)^2 D_j$, a gradient step on the log coupling with $\kappa = 0$.

C EXPERIMENTAL DETAILS

Training. Adam with learning rate 10^{-3} and batch size 32; 100 warm-up steps, then per round 50 local steps and 50 distillation steps (cross-entropy on private data plus KL to the consensus target at temperature 2, weight 1). End-to-end runs use 20 rounds and 20 nodes on an Erdős–Rényi graph with $p = 0.3$, $\alpha = 0.5$, 4 noisy honest nodes (60% uniform label noise) and 20% Byzantine nodes. All 20 models train as one grouped network on a single laptop GPU. **Attacks.** Gaussian: $\mathcal{N}(0, 1)$ entries. Sign flip: $-y_j$. Label flip: y_j with columns reversed. ALIE: $\mu - \sigma$ over honest neighbours. IPM: -0.5μ. Mimic: copy one noisy honest node. Zero-sum: $-\sum_h x_h / b_i$. Dissensus: $x_i - 1.5 \sum_h (x_h - x_i)/b_i$. Targeted: one-hot on class 0. Adaptive: $\mu + s\,d$ for $s \in \{0.25, \dots, 8\}$ and $d \in \{-\sigma,\ \text{flip}(\mu) - \mu,\ -\mu,\ \sigma \odot \xi\}$, maximizing the ℓ_1 change of the frozen rule's output relative to a reference with honest-mean messages. Drift: per public item, $x_i + 0.9\, r_{\min}\, (e_0 - x_i)/\|e_0 - x_i\|$. Byzantine nodes use the attack against every receiver at every iteration. **Tuning grids** (design suite only): BRIDGE-T trim fraction $\{0.1, 0.2, 0.3, 0.4\}$; Krum $f \in \{0.1, 0.25, 0.4\}$; ClippedGossip radius quantile $\{0.25, 0.5, 0.75\}\times$ multiplier $\{0.5, 1, 2\}$; BALANCE $\gamma \in \{0.3, 0.5, 1, 1.5\}$, $\lambda \in \{0, 1\}$, $\alpha \in \{0.3, 0.5\}$; Huber-IRLS iterations $\{5, 10, 20\}$. ARCO grid: loss {Huber, log}$\times$ centre {self, median} $\times\, q \in \{0.25, 0.5\} \times c \in \{0.5, 1, 2\} \times \kappa \in \{0, 0.1\}$ (48 points).

D ADDITIONAL RESULTS

The complete per-attack, per-topology, per-fraction and per-round results are in the following sections.

E REPRODUCIBILITY AND ETHICS

All code, raw results, the pre-registration, all 24 LLM prompts and responses, the 96 agent-written and 200 random programs, and an extended version with per-scenario tables and full end-to-end results are at https://anonymous.4open.science/r/code_for_www27-0ECE/. The experiments use only public benchmark datasets and involve no human subjects. No data were collected from live Web services or Fediverse instances.

F COMPLETE HELD-OUT RESULTS

This section reports every frozen rule on every held-out suite, broken down by attack, topology, Byzantine fraction and FL round. All numbers are relative accuracies, i.e., honest consensus accuracy divided by the accuracy of the oracle consensus over honest nodes. Values above 1 are possible because the oracle includes the four noisy honest nodes, which a robust rule may partly discount. Bold marks the best value in each column. Rows are grouped as in Table 2.

Unseen attacks (Table 7). On FMNIST knowledge, most robust rules handle IPM, mimic and zero-sum well. The attacks that separate rules are the targeted one-hot attack and the adaptive attack. These are the two attacks whose messages are valid probability matrices.

Unseen data (Table 8). On CIFAR-10 and FMNIST with $\alpha = 0.1$, label flip and targeted messages are the hardest attacks for most rules. Gaussian and sign-flip messages are easy for every rule except averaging, because they leave the simplex and are far from honest knowledge.

Topology (Tables 9 and 10). The sparse Erdős–Rényi graph ($p = 0.15$) is the hardest topology for every rule except averaging. It has the fewest edges, so honest nodes have few neighbours and a single Byzantine neighbour carries a large share of each update. On it, the minimum falls to 0.25 for ARCO-B and to 0.39 for BALANCE, while ARCO-G keeps the best minimum (0.70). This is consistent with Corollary 5.3, in which a poorly connected honest graph leaves more room for attackers at any fixed anchor.

Byzantine fraction and placement (Tables 11 and 12). With random placement, the rules separate beyond 20%. At 40%, BALANCE (0.89), LLM run A (0.83) and ARCO-G (0.82) hold best, ClippedGossip falls to 0.72 and ARCO-B to 0.70, and GeoMed and the coordinate-wise rules fall below 0.6. Localized placement is easier than random placement for every rule at every fraction, except averaging at 40%, because the Byzantine cluster borders only part of the honest graph.

FL round (Table 13). Round-1 knowledge is the hardest input. Honest nodes disagree strongly, so a scale chosen from neighbour distances is large and admits more attack mass. The effect is strongest on Web text. There, round-1 relative accuracy ranges from 0.37 (CW-Median) to 0.85 (BALANCE), while at round 10 every rule except averaging reaches 0.92–0.99.

Web text (Table 14). The 20 Newsgroups suite reproduces the image results with lower absolute values. The ordering of ARCO-G (best benign, weakest drift among robust rules) and ARCO-B (best drift) is the same as on images.

Table 7: Suite `heldout_attack` (FMNIST knowledge, unseen attacks, 36 scenarios): relative accuracy per attack, mean over FL rounds $\{1, 4, 10\}$ and two Byzantine placements.

Rule	ipm†	mimic†	zerosum†	dissensus†	targeted†	adaptive†	mean_{att}	min_{att}
Hand-designed (tuned on design suite)								
Mean	**1.000**	0.988	0.314	0.576	0.129	0.551	0.593	0.126
CW-Median	0.943	0.961	0.943	0.925	0.884	0.959	0.936	0.689
BRIDGE-T	0.906	0.955	0.882	0.941	0.824	0.916	0.904	0.653
Krum	0.863	0.850	0.860	0.903	0.885	0.870	0.872	0.640
GeoMed	0.966	0.978	0.964	0.938	0.908	0.969	0.954	0.742
Huber (IRLS)	0.920	0.977	0.911	0.945	0.783	0.943	0.913	0.425
ClippedGossip	0.989	0.986	0.987	0.933	0.954	0.977	0.971	0.846
BALANCE	**1.001**	**0.993**	**1.000**	0.940	0.917	0.976	0.971	0.863
Automated search								
Random search (24)	0.973	0.948	0.974	0.953	0.943	0.916	0.951	0.852
Random search (200)	0.990	0.978	0.989	0.826	0.946	0.864	0.932	0.686
LLM run A	0.993	0.984	0.993	0.899	**0.988**	**0.993**	0.975	0.808
LLM run B	1.000	0.974	1.000	**0.981**	0.954	0.966	**0.979**	0.883
LLM run C	0.969	0.970	0.969	0.905	0.939	0.965	0.953	0.809
LLM run H (small)	0.949	0.976	0.968	0.977	0.966	0.908	0.957	0.846
ARCO (ours, frozen)								
ARCO-G	0.988	0.983	0.989	0.966	0.981	0.958	0.977	**0.914**
ARCO-B	0.997	0.987	0.995	0.919	0.899	0.905	0.950	0.723
ARCO-H	0.996	0.986	0.996	0.950	0.883	0.948	0.960	0.712

Table 8: Suite `heldout_data` (CIFAR-10 and FMNIST-α=0.1 knowledge, 132 scenarios): relative accuracy per attack. †: attack never seen by any designer.

Rule	none	gaussian	signflip	labelflip	alie	ipm†	mimic†	zerosum†	dissensus†	targeted†	adaptive†	mean_{att}	min_{att}
Hand-designed (tuned on design suite)													
Mean	0.857	0.282	0.162	0.091	1.000	0.989	**0.962**	0.354	0.533	0.152	0.451	0.498	0.011
CW-Median	0.955	0.952	0.884	0.822	0.904	0.884	0.896	0.884	0.888	0.789	0.932	0.884	0.309
BRIDGE-T	0.967	0.952	0.940	0.869	0.965	0.945	0.868	0.933	0.922	0.841	0.955	0.919	0.577
Krum	0.852	0.845	0.833	0.822	0.840	0.807	0.684	0.840	0.822	0.817	0.807	0.812	0.338
GeoMed	0.982	1.002	0.905	0.906	0.996	0.993	0.879	0.989	0.937	0.816	0.971	0.940	0.416
Huber (IRLS)	0.983	1.001	0.835	0.842	**1.003**	0.975	0.936	0.931	0.978	0.702	0.935	0.914	0.230
ClippedGossip	0.996	0.981	0.902	0.913	0.984	0.979	0.958	0.977	0.912	0.856	0.969	0.943	0.506
BALANCE	0.943	0.998	**0.995**	0.837	1.000	0.987	0.957	**0.998**	0.959	**0.977**	0.987	**0.970**	0.307
Automated search													
Random search (24)	0.943	0.948	0.936	0.913	0.907	0.942	0.890	0.944	0.930	0.926	0.882	0.922	**0.785**
Random search (200)	0.969	0.955	0.969	0.910	0.965	0.975	0.946	0.975	0.649	0.896	0.891	0.913	0.482
LLM run A	**1.009**	0.988	0.988	**0.970**	0.986	0.988	0.936	0.988	0.889	0.942	0.975	0.965	0.671
LLM run B	0.961	0.989	0.984	0.956	0.969	0.984	0.876	0.984	**0.979**	0.904	**0.987**	0.961	0.571
LLM run C	0.965	0.950	0.940	0.888	0.950	0.940	0.893	0.940	0.844	0.875	0.935	0.916	0.615
LLM run H (small)	1.003	0.985	0.958	0.966	0.965	0.944	0.934	0.972	0.973	0.961	0.917	0.958	0.696
ARCO (ours, frozen)													
ARCO-G	1.004	0.974	0.955	0.965	0.971	0.970	0.949	0.972	0.959	0.960	0.950	0.963	0.770
ARCO-B	0.967	**1.005**	0.919	0.770	0.999	**0.997**	**0.962**	0.995	0.912	0.750	0.883	0.919	0.369
ARCO-H	0.992	1.002	0.902	0.888	0.998	0.996	0.959	0.995	0.961	0.790	0.936	0.943	0.390

Table 9: Suite `heldout_topo` (CIFAR-10 knowledge at round 4 on ring, Watts–Strogatz, random geometric and sparse ER graphs, 88 scenarios): relative accuracy per attack, mean over the four topologies.

Rule	none	gaussian	signflip	labelflip	alie	ipm†	mimic†	zerosum†	dissensus†	targeted†	adaptive†	mean_{att}	min_{att}
Hand-designed (tuned on design suite)													
Mean	0.802	0.354	0.266	0.218	**0.964**	**0.963**	**0.938**	0.408	0.560	0.214	0.548	0.543	0.113
CW-Median	0.862	0.845	0.819	0.723	0.871	0.836	0.837	0.828	0.817	0.763	0.857	0.820	0.594
BRIDGE-T	0.872	0.848	0.812	0.733	0.886	0.839	0.860	0.829	0.831	0.754	0.861	0.825	0.606
Krum	0.725	0.693	0.674	0.649	0.742	0.661	0.581	0.693	0.709	0.665	0.695	0.676	0.528
GeoMed	0.891	0.879	0.801	0.769	0.912	0.880	0.812	0.867	0.847	0.757	0.862	0.839	0.624
Huber (IRLS)	0.890	0.898	0.723	0.669	0.941	0.886	0.910	0.841	0.867	0.630	0.850	0.821	0.327
ClippedGossip	0.915	0.904	0.841	0.820	0.932	0.925	0.910	0.902	0.860	0.805	0.886	0.878	0.604
BALANCE	0.841	**0.956**	**0.956**	0.703	0.953	0.958	0.938	**0.956**	0.882	**0.929**	**0.929**	**0.916**	0.386
Automated search													
Random search (24)	0.857	0.844	0.851	0.798	0.855	0.877	0.834	0.870	0.837	0.821	0.801	0.839	0.621
Random search (200)	0.902	0.873	0.894	0.847	0.912	0.917	0.918	0.912	0.632	0.828	0.779	0.851	0.553
LLM run A	**0.923**	0.916	0.935	0.860	0.929	0.935	0.891	0.935	0.838	0.878	0.905	0.902	0.643
LLM run B	0.872	0.915	0.914	0.844	0.906	0.914	0.843	0.914	0.884	0.810	0.877	0.882	0.668
LLM run C	0.888	0.889	0.888	0.805	0.899	0.888	0.854	0.888	0.811	0.822	0.873	0.861	0.666
LLM run H (small)	0.907	0.868	0.854	0.856	0.901	0.874	0.878	0.874	**0.885**	0.867	0.834	0.869	0.697
ARCO (ours, frozen)													
ARCO-G	**0.923**	0.882	0.875	**0.870**	0.913	0.914	0.902	0.912	0.882	0.869	0.882	0.890	**0.701**
ARCO-B	0.903	0.898	0.847	0.682	0.957	0.952	0.929	0.943	0.857	0.699	0.845	0.861	0.247
ARCO-H	0.907	0.898	0.834	0.734	0.948	0.944	0.910	0.939	0.871	0.713	0.854	0.864	0.371

Table 10: Per-topology relative accuracy (suite `heldout_topo`, attacked scenarios): mean and minimum over the 10 active attacks and two placements.

Rule	ring		ws		geo		er sparse	
	mean	min	mean	min	mean	min	mean	min
Hand-designed (tuned on design suite)								
Mean	0.536	0.158	0.515	0.113	0.533	0.140	0.589	0.113
CW-Median	0.802	0.701	0.871	0.699	0.860	0.755	0.746	0.594
BRIDGE-T	0.802	0.701	0.856	0.606	0.888	0.755	0.754	0.609
Krum	0.678	0.589	0.708	0.528	0.683	0.528	0.635	0.575
GeoMed	0.840	0.761	0.874	0.667	0.883	0.633	0.757	0.624
Huber (IRLS)	0.820	0.666	0.799	0.327	0.887	0.560	0.780	0.544
ClippedGossip	0.877	0.789	0.931	0.816	0.897	0.829	0.809	0.604
BALANCE	**0.920**	0.753	0.954	0.676	**0.945**	0.775	**0.845**	0.386
Automated search								
Random search (24)	0.826	0.751	0.908	0.834	0.870	0.784	0.752	0.621
Random search (200)	0.846	0.622	0.904	0.591	0.874	0.553	0.782	0.565
LLM run A	0.891	**0.803**	**0.965**	0.859	0.913	0.835	0.839	0.643
LLM run B	0.847	0.788	0.950	0.747	0.898	0.782	0.833	0.668
LLM run C	0.844	0.747	0.913	0.814	0.898	0.803	0.791	0.666
LLM run H (small)	0.855	0.784	0.944	0.853	0.893	0.827	0.784	0.697
ARCO (ours, frozen)								
ARCO-G	0.871	0.790	0.961	**0.909**	0.916	**0.876**	0.812	**0.701**
ARCO-B	0.886	0.705	0.904	0.607	0.904	0.794	0.749	0.247
ARCO-H	0.879	0.767	0.904	0.637	0.902	0.783	0.773	0.371

Table 11: Suite `heldout_frac` (CIFAR-10 knowledge at round 4, Byzantine fraction 10–40%, random and localized placement, 176 scenarios): relative accuracy per attack.

Rule	none	gaussian	signflip	labelflip	alie	ipm†	mimic†	zerosum†	dissensus†	targeted†	adaptive†	mean_{att}	min_{att}
Hand-designed (tuned on design suite)													
Mean	0.829	0.329	0.248	0.225	**0.993**	**0.994**	0.939	0.348	0.546	0.183	0.592	0.540	0.068
CW-Median	0.913	0.823	0.768	0.690	0.913	0.842	0.819	0.832	0.852	0.699	0.868	0.811	0.166
BRIDGE-T	0.920	0.765	0.695	0.677	0.946	0.834	0.830	0.810	0.859	0.660	0.762	0.784	0.163
Krum	0.811	0.760	0.742	0.693	0.827	0.682	0.603	0.744	0.775	0.643	0.755	0.722	0.180
GeoMed	0.937	0.914	0.752	0.728	0.930	0.857	0.797	0.888	0.889	0.682	0.919	0.836	0.179
Huber (IRLS)	0.924	0.843	0.656	0.658	0.964	0.857	0.878	0.874	0.866	0.620	0.817	0.803	0.110
ClippedGossip	0.964	0.963	0.865	0.814	0.980	0.965	0.901	0.947	0.881	0.784	0.916	0.902	0.247
BALANCE	0.899	**0.995**	**0.995**	0.650	0.990	**0.995**	**0.944**	**0.995**	0.925	**0.985**	**0.971**	**0.944**	0.158
Automated search													
Random search (24)	0.908	0.897	0.898	0.820	0.908	0.932	0.836	0.919	0.878	0.840	0.831	0.876	0.383
Random search (200)	0.950	0.896	0.934	0.848	0.968	0.964	0.917	0.951	0.625	0.811	0.772	0.869	0.299
LLM run A	0.969	0.975	0.987	0.903	0.971	0.987	0.879	0.987	0.826	0.927	0.936	0.938	**0.500**
LLM run B	0.913	0.964	0.967	0.822	0.965	0.967	0.828	0.967	**0.945**	0.715	0.924	0.906	0.179
LLM run C	0.941	0.933	0.948	0.821	0.955	0.948	0.834	0.948	0.809	0.808	0.904	0.891	0.308
LLM run H (small)	0.949	0.917	0.854	0.837	0.952	0.866	0.857	0.872	0.930	0.810	0.830	0.873	0.180
ARCO (ours, frozen)													
ARCO-G	**0.980**	0.927	0.925	**0.908**	0.969	0.974	0.900	0.975	0.922	0.881	0.926	0.931	0.391
ARCO-B	0.931	0.966	0.875	0.646	**0.993**	0.990	0.893	0.983	0.850	0.682	0.867	0.874	0.151
ARCO-H	0.945	0.953	0.851	0.705	0.987	0.988	0.884	0.985	0.899	0.684	0.901	0.884	0.179

Table 12: Relative accuracy vs. Byzantine fraction and placement (suite `heldout_frac`, mean over the 10 active attacks and two seeds).

Rule	random placement				localized placement			
	10%	20%	30%	40%	10%	20%	30%	40%
Hand-designed (tuned on design suite)								
Mean	0.564	0.507	0.516	0.534	0.615	0.526	0.528	0.526
CW-Median	0.935	0.892	0.639	0.460	0.948	0.915	0.895	0.800
BRIDGE-T	0.962	0.894	0.486	0.447	0.981	0.946	0.924	0.630
Krum	0.793	0.741	0.612	0.548	0.809	0.804	0.778	0.694
GeoMed	0.977	0.907	0.624	0.550	0.990	0.953	0.926	0.758
Huber (IRLS)	0.985	0.888	0.539	0.480	**0.999**	0.963	0.920	0.652
ClippedGossip	0.982	0.936	0.803	0.718	0.989	0.971	0.948	0.865
BALANCE	0.987	0.949	**0.873**	**0.891**	0.997	**0.982**	**0.970**	0.906
Automated search								
Random search (24)	0.923	0.909	0.789	0.719	0.940	0.922	0.923	0.883
Random search (200)	0.953	0.903	0.744	0.693	0.960	0.930	0.921	0.846
LLM run A	0.985	0.955	0.872	0.832	0.991	0.978	0.969	0.919
LLM run B	0.974	0.942	0.812	0.720	**0.998**	0.976	0.953	0.876
LLM run C	0.946	0.904	0.811	0.763	0.960	0.941	0.926	0.874
LLM run H (small)	0.984	0.942	0.741	0.540	0.994	0.960	0.942	0.878
ARCO (ours, frozen)								
ARCO-G	0.980	**0.959**	0.858	0.824	0.980	0.970	0.950	**0.926**
ARCO-B	**0.988**	0.914	0.729	0.697	0.990	0.963	0.940	0.773
ARCO-H	0.985	0.928	0.737	0.705	0.994	0.973	0.944	0.803

Table 13: Relative accuracy by FL round of the knowledge snapshot (attacked scenarios). Early knowledge (round 1) is weak and heterogeneous; late knowledge (round 10) is strong and similar.

Rule	CIFAR-10			FMNIST-0.1			20 Newsgroups		
	r1	r4	r10	r1	r4	r10	r1	r4	r10
Hand-designed (tuned on design suite)									
Mean	0.507	0.507	0.522	0.458	0.499	0.491	0.404	0.432	0.440
CW-Median	0.781	0.892	0.968	0.674	0.978	1.009	0.371	0.893	0.941
BRIDGE-T	0.821	0.894	0.950	0.878	0.978	0.993	0.510	0.901	0.933
Krum	0.689	0.741	0.895	0.645	0.924	0.977	0.411	0.811	0.923
GeoMed	0.870	0.907	0.970	0.880	**1.010**	1.000	0.719	0.906	0.946
Huber (IRLS)	0.859	0.888	0.949	0.859	0.949	0.978	0.727	0.886	0.941
ClippedGossip	0.902	0.936	0.981	0.864	0.975	0.999	0.763	0.944	0.989
BALANCE	0.930	0.949	0.965	**0.959**	1.010	1.005	**0.847**	0.929	0.934
Automated search									
Random search (24)	0.855	0.909	0.963	0.876	0.949	0.977	0.591	0.908	0.957
Random search (200)	0.884	0.903	0.945	0.829	0.950	0.966	0.695	0.903	0.959
LLM run A	**0.933**	0.955	0.978	0.932	0.994	0.997	0.844	**0.972**	**0.992**
LLM run B	0.920	0.942	0.986	0.895	1.006	**1.018**	0.695	0.966	0.978
LLM run C	0.845	0.904	0.964	0.837	0.961	0.983	0.623	0.934	0.982
LLM run H (small)	0.920	0.942	0.975	0.912	0.990	1.006	0.834	0.935	0.951
ARCO (ours, frozen)									
ARCO-G	0.931	**0.959**	**0.989**	0.906	0.993	0.998	0.820	0.956	0.975
ARCO-B	0.867	0.914	0.972	0.872	0.927	0.964	0.776	0.930	0.975
ARCO-H	0.891	0.928	0.975	0.891	0.977	0.995	0.779	0.939	0.972

Table 14: Web-text suite (20 Newsgroups knowledge, 20 classes, 72 scenarios): relative accuracy per attack, mean over FL rounds $\{1, 4, 10\}$ and two placements.

Rule	none	gaussian	signflip	labelflip	alie	ipm†	mimic†	zerosum†	dissensus†	targeted†	adaptive†	drift†	mean_{att}	min_{att}
Hand-designed (tuned on design suite)														
Mean	0.740	0.128	0.126	0.086	0.975	0.977	**0.928**	0.340	0.412	0.065	0.214	0.764	0.456	0.048
CW-Median	0.799	0.798	0.722	0.691	0.769	0.722	0.779	0.722	0.742	0.660	0.747	0.748	0.736	0.120
BRIDGE-T	0.841	0.850	0.771	0.729	0.835	0.779	0.809	0.777	0.789	0.676	0.798	0.772	0.780	0.183
Krum	0.728	0.732	0.741	0.674	0.731	0.695	0.684	0.733	0.727	0.724	0.707	0.613	0.706	0.244
GeoMed	0.885	0.917	0.856	0.799	0.937	0.905	0.797	0.902	0.862	0.705	0.891	0.746	0.847	0.210
Huber (IRLS)	0.862	0.927	0.822	0.722	0.977	0.923	0.903	0.869	0.897	0.605	0.870	0.793	0.846	0.084
ClippedGossip	0.914	0.952	0.895	0.844	0.955	0.957	0.898	0.938	0.866	0.751	0.932	0.793	0.889	0.283
BALANCE	0.833	0.967	0.967	0.482	0.964	0.965	0.919	0.967	0.893	**0.967**	**0.942**	0.821	0.896	0.067
Automated search														
Random search (24)	0.839	0.830	0.844	0.790	0.799	0.849	0.791	0.856	0.826	0.827	0.771	0.673	0.805	0.206
Random search (200)	0.902	0.874	0.921	0.825	0.927	0.938	0.919	0.927	0.625	0.714	0.855	0.787	0.846	0.156
LLM run A	0.933	0.968	**0.974**	0.893	0.953	0.974	0.881	0.974	0.866	0.941	0.940	0.725	**0.917**	0.244
LLM run B	0.848	0.939	0.945	0.819	0.946	0.945	0.820	0.945	0.895	0.670	0.870	0.677	0.861	0.067
LLM run C	0.872	0.908	0.893	0.813	0.854	0.893	0.815	0.893	0.781	0.760	0.851	0.751	0.838	0.257
LLM run H (small)	0.942	0.913	0.901	0.887	0.932	0.925	0.890	0.917	**0.923**	0.914	0.862	0.757	0.893	**0.351**
ARCO (ours, frozen)														
ARCO-G	**0.946**	0.893	0.932	**0.918**	0.938	0.954	0.909	0.955	0.898	0.888	0.885	0.697	0.897	0.176
ARCO-B	0.899	**0.985**	0.930	0.782	**0.978**	**0.980**	0.906	**0.977**	0.840	0.735	0.825	**0.853**	0.890	0.129
ARCO-H	0.899	0.972	0.917	0.802	0.977	0.978	0.897	**0.977**	0.891	0.668	0.887	0.791	0.887	0.140

G ACCUMULATION, ANCHOR AND BOUND: FULL RESULTS

Table 15 gives the numbers behind Fig. 4. Under drift, ARCO-G falls from 0.80 at $T = 5$ to 0.73 at $T = 160$, the accumulation predicted by Proposition 5.4, whereas ARCO-B stays between 0.93 and 0.96. With stubborn Byzantine nodes, the anchor-free rules that accept them converge towards them: BALANCE falls from 0.95 to 0.80 and averaging from 0.90 to 0.83. The anchored ARCO configurations do not fall. Table 16 gives the full anchor sweep behind Fig. 3, and Table 17 the 18 tightness measurements of Theorem 5.1.

Table 15: Relative accuracy vs. number of consensus iterations T under the drift attack and with stubborn (*none*) Byzantine nodes (held-out data; mean over datasets, rounds and seeds).

Rule	drift						none (stubborn)					
	T=5	T=10	T=20	T=40	T=80	T=160	T=5	T=10	T=20	T=40	T=80	T=160
ARCO-G	0.802	0.774	0.765	0.759	0.739	0.725	0.989	1.001	1.002	1.000	0.998	1.004
ARCO-B	0.955	0.959	0.958	0.950	0.941	0.933	0.958	0.962	0.964	0.967	0.967	0.965
ARCO-H	0.917	0.906	0.893	0.877	0.862	0.853	0.988	0.987	0.989	0.990	0.991	0.990
ClippedGossip	0.884	0.902	0.885	0.872	0.867	0.865	0.977	0.994	0.989	0.988	0.987	0.987
BALANCE	0.942	0.954	0.953	0.950	0.949	0.949	0.949	0.913	0.851	0.821	0.809	0.802
GeoMed	0.897	0.882	0.876	0.875	0.874	0.874	0.978	0.978	0.979	0.978	0.979	0.980
BRIDGE-T	0.908	0.909	0.904	0.903	0.903	0.902	0.979	0.973	0.971	0.970	0.970	0.970
Mean	0.951	0.949	0.946	0.944	0.945	0.945	0.897	0.853	0.830	0.825	0.825	0.825
LLM run A	0.864	0.849	0.846	0.848	0.850	0.848	0.999	1.004	1.004	1.004	1.003	1.003

Table 16: Anchor sweep: relative accuracy per attack and honest disagreement (mean squared distance between honest targets) vs. κ ($T = 40$, held-out data). Only κ is changed in the frozen configurations.

Configuration, attack	κ=0	κ=0.02	κ=0.05	κ=0.1	κ=0.2	κ=0.5	κ=1	κ=2
ARCO-B, drift	0.943	0.945	0.950	0.947	0.946	0.939	0.907	0.850
ARCO-B, none	0.963	0.965	0.969	0.971	0.966	0.967	0.956	0.907
ARCO-B, labelflip	0.738	0.766	0.787	0.799	0.817	0.835	0.826	0.777
ARCO-B, targeted	0.697	0.741	0.800	0.845	0.870	0.868	0.855	0.814
ARCO-B, disagreement	0.0521	0.0540	0.0535	0.0554	0.0600	0.0782	0.1089	0.1549
ARCO-G, drift	0.680	0.707	0.721	0.737	0.760	0.790	0.769	0.737
ARCO-G, none	1.019	1.019	1.010	0.998	0.973	0.895	0.827	0.768
ARCO-G, labelflip	0.988	0.989	0.983	0.978	0.950	0.871	0.805	0.753
ARCO-G, targeted	0.971	0.980	0.979	0.976	0.952	0.871	0.807	0.756
ARCO-G, disagreement	0.0320	0.0366	0.0446	0.0566	0.0815	0.1409	0.2033	0.2702

Table 17: Tightness of Theorem 1: measured deviation $\|X^m - X^\star\|$ of the converged attacked solution from the attack-free one vs. the bound $\delta\|b\|_2/\kappa$ (projected gradient, absolute-scale Huber coupling, 3000 iterations, CIFAR-10 knowledge on 200 public items).

Attack	κ	δ	deviation	bound	ratio
gaussian	0.05	0.5	0.818	68.557	1.2%
signflip	0.05	0.5	25.927	68.557	37.8%
drift	0.05	0.5	54.614	68.557	79.7%
gaussian	0.05	2	3.111	274.226	1.1%
signflip	0.05	2	32.467	274.226	11.8%
drift	0.05	2	54.711	274.226	20.0%
gaussian	0.2	0.5	0.346	17.139	2.0%
signflip	0.2	0.5	9.198	17.139	53.7%
drift	0.2	0.5	14.370	17.139	83.8%
gaussian	0.2	2	1.325	68.557	1.9%
signflip	0.2	2	25.363	68.557	37.0%
drift	0.2	2	45.479	68.557	66.3%
gaussian	1	0.5	0.308	3.428	9.0%
signflip	1	0.5	2.227	3.428	65.0%
drift	1	0.5	3.150	3.428	91.9%
gaussian	1	2	0.765	13.711	5.6%
signflip	1	2	7.499	13.711	54.7%
drift	1	2	11.507	13.711	83.9%

H DESIGN-SIDE MAP

Table 18 lists the design-side scores of all 60 rules in Fig. 1(a). These are the only numbers used to choose ARCO-G (maximum validation score), ARCO-B (maximum of the mean of validation score and drift at $T = 40$) and ARCO-H (the Huber configuration with the maximum validation score). In 17 of the 24 pairs, the anchored configuration ($\kappa = 0.1$) is more robust to drift at $T = 40$ than its anchor-free twin. In 15 of 24 pairs, centring the scale on the coordinate-wise median instead of on the receiver lowers the validation score.

Table 18: Design-side map used for Fig. 1(a) and for selecting the frozen ARCO configurations: all 48 ARCO grid points (loss, centre, quantile q, scale c, anchor κ) and the 12 other rules. val = validation score; drift and none on FMNIST design data.

Rule	val. score	val. mean	drift T=10	drift T=40	none T=40
huber self q0.25 d0.5 k0.0	0.777	0.835	0.944	0.909	0.938
huber self q0.25 d0.5 k0.1	0.791	0.840	0.951	0.926	0.936
huber self q0.25 d1.0 k0.0	0.754	0.825	0.948	0.923	0.931
huber self q0.25 d1.0 k0.1	0.773	0.832	0.953	0.929	0.927
huber self q0.25 d2.0 k0.0	0.396	0.698	0.983	0.968	0.847
huber self q0.25 d2.0 k0.1	0.453	0.734	0.985	0.973	0.849
huber self q0.5 d0.5 k0.0	0.763	0.816	0.945	0.914	0.925
huber self q0.5 d0.5 k0.1	0.771	0.818	0.954	0.928	0.929
huber self q0.5 d1.0 k0.0	0.661	0.792	0.968	0.953	0.895
huber self q0.5 d1.0 k0.1	0.706	0.798	0.971	0.950	0.910
huber self q0.5 d2.0 k0.0	0.296	0.577	0.985	0.969	0.793
huber self q0.5 d2.0 k0.1	0.324	0.616	**0.987**	**0.975**	0.811
huber median q0.25 d0.5 k0.0	0.754	0.822	0.938	0.913	0.930
huber median q0.25 d0.5 k0.1	0.769	0.825	0.943	0.922	0.935
huber median q0.25 d1.0 k0.0	0.735	0.816	0.935	0.917	0.925
huber median q0.25 d1.0 k0.1	0.743	0.819	0.939	0.925	0.928
huber median q0.25 d2.0 k0.0	0.479	0.748	0.970	0.964	0.875
huber median q0.25 d2.0 k0.1	0.546	0.767	0.971	0.964	0.879
huber median q0.5 d0.5 k0.0	0.743	0.812	0.936	0.919	0.919
huber median q0.5 d0.5 k0.1	0.764	0.816	0.941	0.921	0.930
huber median q0.5 d1.0 k0.0	0.674	0.793	0.950	0.942	0.898
huber median q0.5 d1.0 k0.1	0.712	0.799	0.952	0.938	0.912
huber median q0.5 d2.0 k0.0	0.320	0.624	0.974	0.965	0.794
huber median q0.5 d2.0 k0.1	0.354	0.663	0.974	0.966	0.817
log self q0.25 d0.5 k0.0	0.821	0.850	0.855	0.758	0.972
log self q0.25 d0.5 k0.1	0.808	0.841	0.883	0.830	0.968
log self q0.25 d1.0 k0.0	0.812	0.851	0.881	0.844	0.957
log self q0.25 d1.0 k0.1	0.812	0.850	0.892	0.832	0.963
log self q0.25 d2.0 k0.0	0.709	0.824	0.980	0.964	0.926
log self q0.25 d2.0 k0.1	0.754	0.829	0.982	0.964	0.947
log self q0.5 d0.5 k0.0	**0.822**	0.850	0.854	0.776	0.974
log self q0.5 d0.5 k0.1	**0.822**	0.848	0.880	0.843	**0.984**
log self q0.5 d1.0 k0.0	0.763	0.818	0.943	0.933	0.945
log self q0.5 d1.0 k0.1	0.771	0.819	0.944	0.923	0.960
log self q0.5 d2.0 k0.0	0.469	0.761	0.983	0.966	0.824
log self q0.5 d2.0 k0.1	0.605	0.776	0.986	0.972	0.844
log median q0.25 d0.5 k0.0	0.795	0.831	0.919	0.884	0.948
log median q0.25 d0.5 k0.1	0.794	0.826	0.924	0.889	0.954
log median q0.25 d1.0 k0.0	0.795	0.836	0.902	0.884	0.948
log median q0.25 d1.0 k0.1	0.796	0.836	0.907	0.872	0.957
log median q0.25 d2.0 k0.0	0.728	0.815	0.958	0.952	0.943
log median q0.25 d2.0 k0.1	0.733	0.819	0.960	0.934	0.955
log median q0.5 d0.5 k0.0	0.792	0.833	0.908	0.879	0.947
log median q0.5 d0.5 k0.1	0.798	0.831	0.916	0.886	0.957
log median q0.5 d1.0 k0.0	0.759	0.816	0.928	0.918	0.942
log median q0.5 d1.0 k0.1	0.765	0.817	0.930	0.918	0.956
log median q0.5 d2.0 k0.0	0.667	0.788	0.966	0.963	0.892
log median q0.5 d2.0 k0.1	0.702	0.794	0.967	0.956	0.934
Mean	0.191	0.376	0.987	0.888	0.820
CW-Median	0.747	0.817	0.928	0.924	0.941
BRIDGE-T	0.701	0.782	0.928	0.902	0.931
Krum	0.688	0.770	0.795	0.756	0.901
GeoMed	0.764	0.822	0.918	0.908	0.926
ClippedGossip	0.805	0.843	0.948	0.928	0.942
BALANCE	0.731	0.831	**0.987**	0.969	0.820
Huber (IRLS)	0.665	0.790	0.941	0.929	0.914
LLM run A	0.811	**0.852**	0.880	0.829	0.955
LLM run B	0.799	0.847	0.942	0.926	0.939
LLM run C	0.798	0.836	0.932	0.926	0.950
LLM run H (small)	0.804	0.836	0.894	0.891	0.957

I THE LLM-AGENT STUDY IN DETAIL

Protocol. Each run is an independent sequence of six LLM calls. The agent was a subagent with no tool access beyond reading its prompt file and writing its response file. It never saw the code base, the held-out suites, the pre-registration, or other runs. The round-1 prompt (Listing 2) specifies the interface `aggregate(x_self, X_nb, t, T, state)`, the threat model, the knowledge format, the design suite, and four tuned baselines with their design-suite scores. From round 2 on, the prompt also contains the five best previous programs with their per-attack design scores (Listing 3). Each response must contain four programs, which we extract, compile, and score on the design and validation suites. All 96 programs compiled and ran without errors. Runs A, B and C use Claude Opus 5.5 and run H uses Claude Haiku 5.5.

Every program (Tables 19–22). For each run we list every program with its round, design score (the feedback), validation score (the selection criterion) and held-out-lite accuracy (CIFAR-10 and FMNIST-0.1 knowledge at round 4, all 11 attacks, never shown to the agent). Fig. 6 plots design score against held-out-lite accuracy for all 296 automatically designed programs. Over the 200 random programs, the two are well correlated (Spearman 0.74). Within each LLM run, which concentrates on high design scores, the correlation is weak, so further design-score gains are not reliable held-out gains.

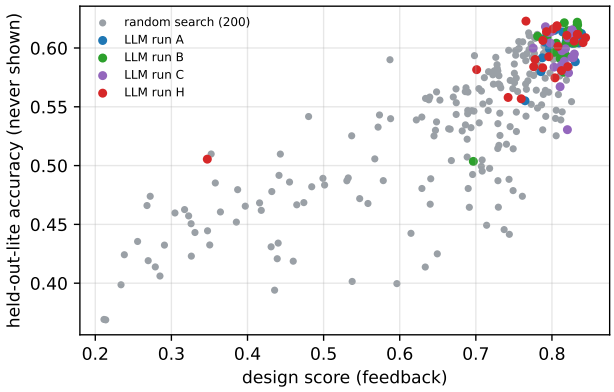


Figure 6: Design score (the agent's feedback) vs. held-out-lite accuracy for all 200 random-search programs and all 96 LLM-agent programs.

Random-search grammar. Each random program samples a centre (receiver, coordinate-wise median, mean, or trimmed mean). It then samples a dissimilarity to that centre: at node level, ℓ_2, ℓ_1, cosine, ℓ_∞ or argmax disagreement; at row level, ℓ_2, ℓ_1 or ℓ_∞. Next it samples a weighting rule: a hard threshold, an exponential soft weight, clipping, or top-k by rank. Continuous parameters are the threshold multiplier, temperature, top-k fraction, trim fraction, self weight, an optional memory, and the step size. Finally, it chooses whether to project the output onto the simplex. The grammar is written in Python, and every sampled program is released.

Table 19: LLM design run A: every program, with the round it was proposed in, its design score (the feedback the agent saw), validation score (the selection criterion) and held-out-lite accuracy (never shown).

Program	round	design	val.	held-out-lite
A1.1_HellingerGeoMedian	1	0.814	0.770	0.614
A1.2_AgreementTrust	1	0.816	0.721	0.587
A1.3_PerSampleTrimmedHellinger	1	0.765	0.716	0.555
A1.4_ReputationClippedGossip	1	0.836	**0.811**	0.605
A2.1_HellingerContractiveRepClip	2	0.827	0.802	0.604
A2.2_IterativeCenteredClipRep	2	0.792	0.770	0.608
A2.3_SoftKrumReputationGossip	2	0.787	0.718	0.592
A2.4_RowwiseClipNeighbourReputation	2	0.819	0.790	0.605
A3.1_A3_1_SelfAwareCenterRepClip	3	0.836	0.810	0.607
A3.2_A3_2_DualEvidenceRepClip	3	0.831	0.802	0.588
A3.3_A3_3_TrustScaledRadiusClip	3	0.837	0.809	0.610
A3.4_A3_4_DawidSkeneReliabilityGossip	3	0.786	0.747	0.580
A4.1_A4_1_AnnealedSelfTrustRadiusClip	4	0.837	0.809	0.611
A4.2_A4_2_RowwiseTrustClip	4	0.835	0.808	0.606
A4.3_A4_3_PeerCorroboratedRepClip	4	0.798	0.789	0.614
A4.4_A4_4_RelativeHedgeTrustClip	4	**0.839**	0.795	0.610
A5.1_A5_1_HedgeAnnealedClip	5	**0.838**	0.795	0.611
A5.2_A5_2_DecisionAgreementHedge	5	0.791	0.785	0.605
A5.3_A5_3_ReputationWeightedGeoMedian	5	0.797	0.801	0.599
A5.4_A5_4_ConfidenceRowWeightedHedgeClip	5	0.793	0.792	0.608
A6.1_A6_1_TrustedRadiusHedgeClip	6	0.835	0.794	0.611
A6.2_A6_2_IteratedTrustCenteredClip	6	**0.839**	0.796	0.612
A6.3_A6_3_PeerConsensusHedgeClip	6	0.803	0.793	**0.615**
A6.4_A6_4_SampleLevelHedgeClip	6	0.835	0.800	0.604

Table 20: LLM design run B: every program, with the round it was proposed in, its design score (the feedback the agent saw), validation score (the selection criterion) and held-out-lite accuracy (never shown).

Program	round	design	val.	held-out-lite
B1.1_SimplexGeoMedian	1	0.811	0.765	0.591
B1.2_ReputationGossip	1	0.821	0.798	0.615
B1.3_AnchoredCenteredClipping	1	0.794	0.690	0.609
B1.4_ConsistentCore	1	0.819	0.736	0.593
B2.1_B2_1_TrustCore	2	0.833	0.786	**0.622**
B2.2_B2_2_EvidenceLedger	2	0.833	0.789	0.619
B2.3_B2_3_TwoLevelRobustMean	2	0.819	**0.799**	0.609
B2.4_B2_4_SemanticAlignment	2	**0.834**	0.798	0.613
B3.1_R3_1_ReliabilityCore	3	0.810	0.762	0.595
B3.2_R3_2_ClassConditionalReliability	3	0.822	0.792	0.604
B3.3_R3_3_SemanticAlignmentMomentum	3	0.824	0.730	0.609
B3.4_R3_4_LogLinearPooling	3	0.803	0.752	0.596
B4.1_R4_1_SemanticAlignmentRowClip	4	**0.834**	**0.799**	0.614
B4.2_R4_2_CoreAlignmentFusion	4	0.834	0.788	0.620
B4.3_R4_3_PersonalizedPageRankTrust	4	0.818	0.720	0.603
B4.4_R4_4_OneCoinEMWeighting	4	0.816	0.718	0.579
B5.1_R5_1_QualityWeightedRowClip	5	0.830	0.778	0.604
B5.2_R5_2_OverRelaxedTrustGossip	5	0.781	0.731	0.610
B5.3_R5_3_CoreHybridOpinionPool	5	0.816	0.773	**0.621**
B5.4_R5_4_TrustPriorRowHuberIRLS	5	0.833	0.798	0.612
B6.1_TemporalStepTrustRowClip	6	0.833	0.798	0.613
B6.2_ClassConditionalReliability	6	0.806	0.773	0.604
B6.3_RobustDawidSkeneFusion	6	0.697	0.680	0.504
B6.4_AnnealedMixingHuberIRLS	6	0.833	**0.799**	0.613

Table 21: LLM design run C: every program, with the round it was proposed in, its design score (the feedback the agent saw), validation score (the selection criterion) and held-out-lite accuracy (never shown).

Program	round	design	val.	held-out-lite
C1.1_SanRowClip	1	0.775	0.732	0.600
C1.2_ArgmaxReputation	1	0.816	0.716	0.589
C1.3_HellingerGM	1	0.803	**0.798**	0.584
C1.4_RowMADSuspicion	1	0.826	0.781	0.615
C2.1_C2_1_DualSuspicion	2	0.821	0.772	0.609
C2.2_C2_2_HellingerMeanShift	2	0.812	0.786	0.585
C2.3_C2_3_CohesionTrust	2	0.791	0.780	**0.618**
C2.4_C2_4_TrustedReferenceRowMAD	2	0.799	0.762	0.614
C3.1_C3_1_RowMADSuspicionV2	3	0.827	0.791	0.614
C3.2_C3_2_RobustPeerAgreement	3	**0.829**	0.784	0.595
C3.3_C3_3_TemporalDriftSuspicion	3	0.826	0.781	0.615
C3.4_C3_4_RowClippedTrustGossip	3	0.796	0.779	0.613
C4.1_C4_1_PeerAgreementSelfDistrust	4	**0.829**	0.781	0.612
C4.2_C4_2_TukeyIRLSReputation	4	0.813	0.774	0.598
C4.3_C4_3_PersonalizedPageRankTrust	4	0.811	0.743	0.567
C4.4_C4_4_BetaReputationHybrid	4	0.818	0.760	0.599
C5.1_C5_1_WeightedMedianAnchoredAgreement	5	0.828	0.770	0.592
C5.2_C5_2_QualityTemperedReputation	5	0.820	0.758	0.531
C5.3_C5_3_GeoMedianCenteredClipping	5	0.826	0.770	0.592
C5.4_C5_4_LeaveOneOutConsistencyTrimmed	5	0.798	0.713	0.604
C6.1_C6_1_AgreementRepSoftTV	6	**0.829**	0.784	0.607
C6.2_C6_2_ReputationWeightedLabelVote	6	0.827	0.785	0.592
C6.3_C6_3_ContractionTestReputation	6	**0.830**	0.795	0.611
C6.4_C6_4_ReputationIRLSBiweight	6	0.822	0.787	0.579

Table 22: LLM design run H: every program, with the round it was proposed in, its design score (the feedback the agent saw), validation score (the selection criterion) and held-out-lite accuracy (never shown).

Program	round	design	val.	held-out-lite
H1.1_MedoidFilteredMean	1	0.804	0.747	0.575
H1.2_TrustEMAGossip	1	0.742	0.690	0.558
H1.3_GeometricMedianCentre	1	0.807	0.764	0.619
H1.4_RowwiseConsensusWeighted	1	0.796	0.752	0.593
H2.1_HuberIRLSAnchored	2	0.766	0.681	**0.623**
H2.2_AnnealedMedianBand	2	0.820	0.787	0.611
H2.3_SimplexTVReputation	2	0.802	0.771	0.616
H2.4_GeometricMedianFromClip	2	0.808	0.760	0.601
H3.1_AnnealedSoftBandGeoMedian	3	0.829	0.756	0.606
H3.2_LogPoolTrimmedMean	3	0.347	0.317	0.505
H3.3_DirectionGatedClip	3	0.760	0.739	0.557
H3.4_DecayedTrustBand	3	0.788	0.749	0.606
H4.1_HellingerBandMixture	4	0.833	0.803	0.612
H4.2_NeighbourhoodAgreementGeoMed	4	0.778	0.762	0.590
H4.3_PersistentSuspicionBand	4	**0.844**	0.786	0.609
H4.4_CoordinateMADFilteredMean	4	0.776	0.655	0.584
H5.1_H5_1_PersistentBandClippedMix	5	0.841	**0.804**	0.605
H5.2_H5_2_WelschIRLSAnnealed	5	0.793	0.741	0.614
H5.3_H5_3_PerSampleRowTrust	5	0.821	0.779	0.584
H5.4_H5_4_LargestGapCutWithBanMemory	5	0.807	0.790	0.618
H6.1_HellingerPersistentTrust	6	0.833	0.803	0.612
H6.2_HuberIRLSWithBanMemory	6	0.701	0.453	0.582
H6.3_MedoidAnchoredAnnealedTrim	6	0.788	0.641	0.583
H6.4_ConfidenceWeightedBand	6	0.813	0.778	0.581

J END-TO-END TRAINING: FULL RESULTS

Tables 23 and 24 give every end-to-end cell with its spread over seeds, and Figs. 7 and 8 the per-round training curves. On CIFAR-10, the 29 cells with two seeds have a median seed-to-seed difference of 0.5 points and a maximum of 4.1 points. Under sign flip and targeted attacks, averaging collapses within the first rounds and never recovers. The gap between ARCO and the median-type rules has a specific shape on CIFAR-10. Under several attacks (e.g., sign flip for ARCO-B, IPM for ARCO-H), the ARCO configurations reach 33–37% within about five rounds and then plateau near the level of local-only training (32.4%), while the median-type rules keep improving to about 44%. So the gap does not come from slow accumulation over many rounds: ARCO stops benefiting from collaboration early. We do not yet know why this happens in training, when the same configurations produce accurate targets at the consensus level.

On 20 Newsgroups, every cell has three seeds, and the knowledge-sharing methods have a seed standard deviation of at most 5.6 points. The picture is the same as on CIFAR-10: BALANCE and LLM run H have the best attacked means, ARCO-B is the best rule under drift, and ARCO-G is the worst under drift. Across the non-drift attacks, the argmax accuracy of the consensus targets on the public items is similar for ARCO-G (54.7%), LLM run H (54.1%) and BALANCE (53.7%), but the trained students differ by up to 4 points. Target accuracy on the public items is higher than student test accuracy for every rule, because the public items are distilled on in every round. For DFedAvg, the attacks act on parameters rather than on knowledge, so they are not the same attacks: several of them (e.g., ALIE and sign flip) hurt less than stubborn Byzantine nodes do. We therefore report only DFedAvg's benign accuracy in Table 4.

Table 23: End-to-end DFL on CIFAR-10, per attack: honest test accuracy (%), mean ± std over seeds (number of seeds in parentheses where it differs from the table default).

Method	none	signflip	alie	ipm	dissensus	targeted	adaptive	drift
Local only	32.4±0.5 (2)	32.4±0.5 (2)	32.4±0.5 (2)	32.4±0.5 (2)	32.4±0.5 (2)	32.4±0.5 (2)	32.4±0.5 (2)	32.4±0.5 (2)
DFedAvg (param.)	40.8	33.2	10.0	18.8	15.7	–	–	–
Gossip averaging	40.2±0.3 (2)	18.4±1.2 (2)	45.4	43.7	31.2	24.9±0.2 (2)	36.8	41.6±0.1 (2)
CW-Median	44.0	43.4	43.6	43.4	40.7	36.5	43.9	41.8
BRIDGE-T	43.8	43.1	43.9	43.1	41.4	34.6	44.1	41.6
GeoMed	42.0±0.3 (2)	34.6±2.1 (2)	44.8	43.1	39.3	37.5±0.0 (2)	40.3	39.3±0.5 (2)
ClippedGossip	42.2±0.1 (2)	36.7±0.2 (2)	45.1	43.5	41.1	38.9±0.5 (2)	40.7	39.8±0.6 (2)
BALANCE	40.7±0.2 (2)	43.4±0.1 (2)	45.1	43.6	41.6	43.2±0.1 (2)	37.5	41.5±0.2 (2)
Random search (200)	40.9	43.3	44.7	43.3	30.0	40.5	35.2	–
LLM run H	43.3	43.3	44.0	40.1	43.7	43.5	40.2	–
LLM run A	42.5±0.2 (2)	35.3±1.4 (2)	45.7	33.9	35.2	41.4±0.2 (2)	36.2	35.7±1.6 (2)
ARCO-G	42.8±0.2 (2)	39.7±0.7 (2)	38.2	38.9	42.1	41.7±0.0 (2)	37.5	34.5±0.8 (2)
ARCO-B	41.3±0.4 (2)	33.5±0.7 (2)	38.5	34.0	37.4	35.7±0.9 (2)	33.0	41.5±0.1 (2)
ARCO-H	41.8	29.9	38.4	33.8	41.8	35.9	34.4	40.1

Table 24: End-to-end DFL on 20 Newsgroups, per attack: honest test accuracy (%), mean ± std over seeds (number of seeds in parentheses where it differs from the table default).

Method	none	signflip	alie	ipm	dissensus	targeted	adaptive	drift
Local only	21.7±0.0 (3)	21.7±0.0 (3)	21.7±0.0 (3)	21.7±0.0 (3)	21.7±0.0 (3)	21.7±0.0 (3)	21.7±0.0 (3)	21.7±0.0 (3)
DFedAvg (param.)	40.7±3.6 (3)	50.8±0.6 (3)	54.8±2.1 (3)	24.1±5.7 (3)	32.9±4.3 (3)	–	–	–
Gossip averaging	31.4±1.6 (3)	13.5±1.6 (3)	37.1±1.7 (3)	35.4±1.2 (3)	16.8±1.8 (3)	8.8±0.7 (3)	19.3±0.1 (3)	32.5±1.2 (3)
CW-Median	37.2±0.8 (3)	33.9±1.2 (3)	33.7±1.1 (3)	33.9±1.2 (3)	29.6±0.9 (3)	26.6±2.7 (3)	32.8±0.8 (3)	29.7±1.3 (3)
BRIDGE-T	37.5±0.7 (3)	35.3±0.9 (3)	35.0±0.6 (3)	35.5±0.9 (3)	31.6±1.3 (3)	27.8±3.7 (3)	33.6±0.5 (3)	31.1±1.9 (3)
GeoMed	35.3±0.6 (3)	24.1±2.8 (3)	35.4±1.3 (3)	34.8±1.0 (3)	31.8±2.1 (3)	28.8±0.4 (3)	37.4±3.4 (3)	30.5±0.7 (3)
ClippedGossip	35.3±0.6 (3)	24.3±1.4 (3)	36.4±1.7 (3)	35.0±1.1 (3)	32.5±1.1 (3)	29.1±0.4 (3)	36.2±3.2 (3)	31.4±0.6 (3)
BALANCE	31.8±1.5 (3)	35.4±1.1 (3)	36.7±1.6 (3)	35.4±1.1 (3)	32.7±0.9 (3)	35.4±1.1 (3)	35.2±1.9 (3)	32.8±1.1 (3)
Random search (200)	34.7±0.4 (3)	34.9±1.3 (3)	35.9±2.4 (3)	35.0±1.2 (3)	17.3±1.4 (3)	20.6±5.6 (3)	27.1±0.3 (3)	29.2±1.2 (3)
LLM run H	35.8±0.5 (3)	35.1±1.1 (3)	33.7±2.0 (3)	34.3±1.4 (3)	33.8±2.0 (3)	35.2±1.2 (3)	34.7±1.7 (3)	27.0±4.1 (3)
LLM run A	36.1±0.2 (3)	21.2±1.7 (3)	36.3±1.8 (3)	21.2±1.7 (3)	28.8±4.1 (3)	34.3±1.4 (3)	34.6±1.3 (3)	28.1±3.2 (3)
ARCO-G	35.6±0.5 (3)	31.3±1.3 (3)	27.8±0.9 (3)	30.4±0.7 (3)	34.1±1.1 (3)	34.0±1.3 (3)	29.6±2.3 (3)	26.8±2.9 (3)
ARCO-B	34.5±1.1 (3)	24.4±1.0 (3)	28.7±0.8 (3)	24.5±0.6 (3)	31.2±1.6 (3)	31.2±1.6 (3)	26.8±1.0 (3)	33.1±1.0 (3)
ARCO-H	34.7±1.0 (3)	18.0±1.8 (3)	28.0±1.0 (3)	23.5±0.9 (3)	33.7±1.2 (3)	24.4±4.5 (3)	28.5±1.9 (3)	31.8±0.4 (3)

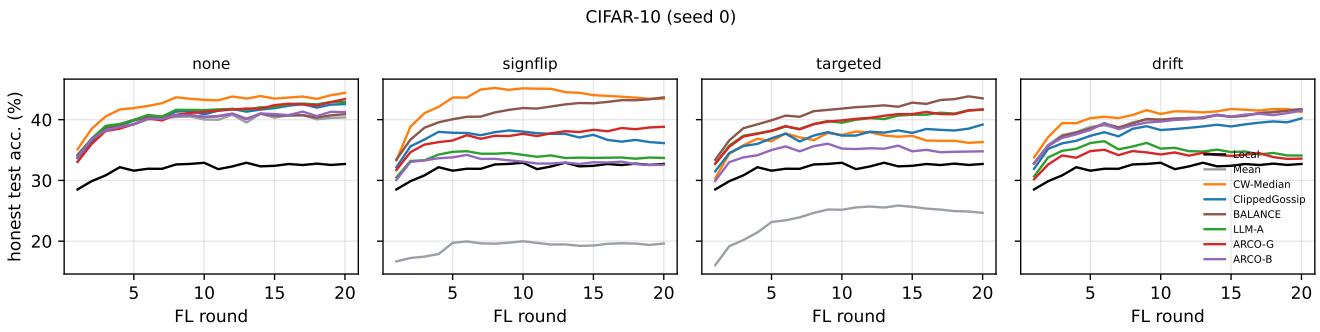


Figure 7: CIFAR-10 end-to-end training curves (honest test accuracy per FL round, seed 0).

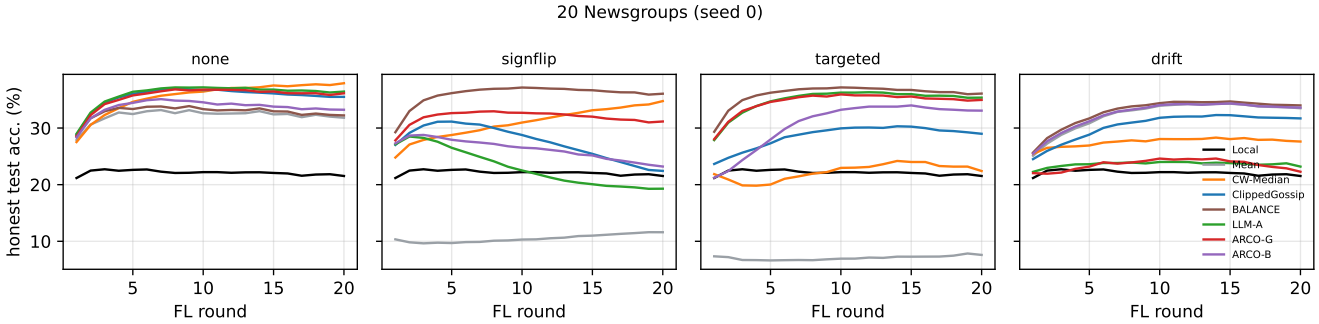


Figure 8: 20 Newsgroups end-to-end training curves (honest test accuracy per FL round, seed 0).

K VERBATIM ARTIFACTS

The listings below are reproduced from the repository and converted to ASCII. Local file paths are replaced by <repo>.

Listing 1: Pre-registration file (verbatim), written before any automated design run.

```
# Pre-registration (written 2026-10-20 06:57 CEST, before any LLM / random-search design run)

**Object being designed**: the per-node gossip aggregation rule `aggregate(x_self, X_nb, t, T, state) -> (m,C)` used for T=10 synchronous consensus iterations on
knowledge matrices (soft predictions on a shared public set, m=1000, C=10).

**Designer feedback (the only signal any designer sees)**: suite `design` = FMNIST (Dirichlet 0.5, 20 nodes, 4 honest noisy nodes w/ 60% label noise) snapshots at FL rounds {1,4,10}
x attacks {none, gaussian, signflip, labelflip, alie} x ER(p=0.3) graph, 20% Byzantine, placement seed 0.
Score = 0.5 * mean accuracy + 0.5 * worst-case accuracy of honest consensus targets on public labels.

**Selection rule (fixed)**: within each design run, the final program is the candidate with highest score on
suite `design_val` (same distribution, Byzantine placement seeds {1,2}). No held-out result is used for selection.

**Held-out evaluation (never shown to any designer)**:
1. `heldout_attack`: ipm, mimic, zerosum, dissensus (He et al. 2022), targeted (one-hot hijack), adaptive
   (black-box query of the frozen rule, Shejwalkar & Houmansadr style).
2. `heldout_data`: CIFAR-10 and FMNIST Dirichlet(0.1) knowledge, all 11 attacks.
3. `heldout_topo`: ring lattice, Watts-Strogatz, random geometric, sparse ER.
4. `heldout_frac`: Byzantine fraction {0.1,0.2,0.3,0.4} x {random, localized} placement.
5. End-to-end DFL on CIFAR-10 (distillation with consensus targets), honest test accuracy.

**Comparisons**: 8 handcrafted decentralized rules with hyper-parameters tuned on `design`
(results/tuned_baselines.json); random program search over a hand-written grammar with (a) the same
evaluation budget as one LLM run (24 programs) and (b) 8x that budget (200 programs), selected by the same rule;
LLM zero-shot (round-1 candidates only); 3 independent LLM runs (consistency); 1 run with a smaller LLM.

**Reporting**: all candidates, prompts and responses are logged in design_runs/. Negative results are reported.
```

Listing 2: Run A, round 1 prompt (verbatim). Later rounds append the five best previous programs with per-attack feedback.

````
You are designing a Byzantine-robust aggregation rule for *fully decentralized* federated learning.

## Setting
- 20 nodes on a sparse peer-to-peer graph, no server. Each node trains a local model on private non-IID data.
- Nodes share *knowledge*, not parameters: their softmax predictions on a shared unlabeled public set,
  a matrix of shape (m, C) = (1000, 10); each row is a probability vector.
- Knowledge consensus runs T = 10 synchronous gossip iterations. At iteration t every honest node i receives
  one (m, C) message from each graph neighbour and replaces its state by the output of YOUR function.
  The final state (clipped to >= 0 and row-normalised) becomes the distillation target of node i.
- Some neighbours are Byzantine: they may send arbitrary arrays (not necessarily probabilities), may collude,
  know the honest messages, and may change behaviour over iterations. A node does not know which neighbours,
  or how many, are Byzantine; overall about 20% of nodes are, but locally it can be more.
- Some honest nodes have very noisy labels (their knowledge is poor but not malicious).
- Node degrees are between ~2 and ~10.

## Interface (pure numpy; `np` and `math` are pre-imported; no other imports)
```python
def aggregate(x_self, X_nb, t, T, state):
    # x_self: np.ndarray (m, C) -- this node's current state (initially its own local knowledge)
    # X_nb:   np.ndarray (k, m, C) -- messages received this iteration (order is fixed per node across t)
    # t, T:   current iteration (0-based) and total iterations
    # state:  dict, persists across the T iterations of this node (empty at t = 0); use it for memory
    # return: np.ndarray (m, C), finite values
```
Constraints: deterministic; < 50 ms per call for k <= 10; must not crash for k = 1.

## Objective
Honest nodes should end with accurate (argmax matches the true label) and mutually consistent targets.
Score = 0.5 * mean accuracy + 0.5 * worst-case accuracy over the evaluation scenarios.
The rule will later be deployed against attacks, datasets and topologies beyond the evaluation scenarios
below, so prefer principled mechanisms over tricks that only exploit the specific attacks listed.

## Evaluation scenarios you get feedback on
FashionMNIST knowledge snapshots from FL rounds {1, 4, 10}, Erd?s-R?nyi graph (p=0.3), 20% Byzantine,
attacks: none (Byzantines behave honestly), gaussian (N(0,1) noise), signflip (negated own knowledge),
labelflip (own knowledge with class order reversed), alie (coordinate-wise mean - 1.0*std of honest messages).
Reference points: average local accuracy of honest nodes before consensus ~ 0.684; accuracy of the
honest-only average (an oracle that knows who is Byzantine) ~ 0.860.


## Existing rules (hand-designed, hyper-parameters already tuned on these scenarios)
### Mean: score=0.2005 (mean=0.393, worst=0.008; none=0.818, gaussian=0.249, signflip=0.019, labelflip=0.017, alie=0.862)
```python
def aggregate(x_self, X_nb, t, T, state):
    return (x_self + X_nb.sum(0)) / (len(X_nb) + 1)
```
### CW-Median: score=0.7831 (mean=0.831, worst=0.736; none=0.840, gaussian=0.848, signflip=0.826, labelflip=0.805, alie=0.834)
```python
def aggregate(x_self, X_nb, t, T, state):
    return np.median(np.concatenate([x_self[None], X_nb]), 0)
```
### ClippedGossip: score=0.8193 (mean=0.847, worst=0.792; none=0.847, gaussian=0.853, signflip=0.833, labelflip=0.848, alie=0.853)
```python
def aggregate(x_self, X_nb, t, T, state):
    # self-centred clipping (He, Karimireddy & Jaggi 2022)
    k = len(X_nb)
    D = X_nb - x_self
    nrm = np.linalg.norm(D.reshape(k, -1), axis=1)
    tau = 0.5 * np.quantile(nrm, 0.25) + 1e-12
    scale = np.minimum(1.0, tau / np.maximum(nrm, 1e-12))
    return x_self + (scale[:, None, None] * D).sum(0) / (k + 1)
```
### BALANCE: score=0.8215 (mean=0.854, worst=0.789; none=0.827, gaussian=0.861, signflip=0.861, labelflip=0.862, alie=0.862)
```python
def aggregate(x_self, X_nb, t, T, state):
    # distance-threshold acceptance (Fang et al., CCS 2024)
    nrm = np.linalg.norm((X_nb - x_self).reshape(len(X_nb), -1), axis=1)
    thr = 1.5 * np.exp(-1.0 * t / T) * np.linalg.norm(x_self)
    acc = X_nb[nrm <= thr]
    if len(acc) == 0:
        return x_self
    return 0.5 * x_self + 0.5 * acc.mean(0)
```


## What to return
Propose 4 *diverse* candidate rules. For each: a short heading `### Candidate <name>`, 1-3 sentences on the idea,
then ONE ```python code block defining `aggregate`. Return nothing else. Do not use any tools.
````

Listing 3: Run A, round 2 prompt (verbatim), showing the feedback format.

```
You are designing a Byzantine-robust aggregation rule for *fully decentralized* federated learning.

## Setting
```

```
- 20 nodes on a sparse peer-to-peer graph, no server. Each node trains a local model on private non-IID data.
- Nodes share *knowledge*, not parameters: their softmax predictions on a shared unlabeled public set,
  a matrix of shape (m, C) = (1000, 10); each row is a probability vector.
- Knowledge consensus runs T = 10 synchronous gossip iterations. At iteration t every honest node i receives
  one (m, C) message from each graph neighbour and replaces its state by the output of YOUR function.
  The final state (clipped to >= 0 and row-normalised) becomes the distillation target of node i.
- Some neighbours are Byzantine: they may send arbitrary arrays (not necessarily probabilities), may collude,
  know the honest messages, and may change behaviour over iterations. A node does not know which neighbours,
  or how many, are Byzantine; overall about 20% of nodes are, but locally it can be more.
- Some honest nodes have very noisy labels (their knowledge is poor but not malicious).
- Node degrees are between ~2 and ~10.

## Interface (pure numpy; `np` and `math` are pre-imported; no other imports)
```python
def aggregate(x_self, X_nb, t, T, state):
    # x_self: np.ndarray (m, C) -- this node's current state (initially its own local knowledge)
    # X_nb:   np.ndarray (k, m, C) -- messages received this iteration (order is fixed per node across t)
    # t, T:   current iteration (0-based) and total iterations
    # state:  dict, persists across the T iterations of this node (empty at t = 0); use it for memory
    # return: np.ndarray (m, C), finite values
```
Constraints: deterministic; < 50 ms per call for k <= 10; must not crash for k = 1.

Objective
Honest nodes should end with accurate (argmax matches the true label) and mutually consistent targets.
Score = 0.5 * mean accuracy + 0.5 * worst-case accuracy over the evaluation scenarios.
The rule will later be deployed against attacks, datasets and topologies beyond the evaluation scenarios
below, so prefer principled mechanisms over tricks that only exploit the specific attacks listed.

Evaluation scenarios you get feedback on
FashionMNIST knowledge snapshots from FL rounds {1, 4, 10}, Erd?s-R?nyi graph (p=0.3), 20% Byzantine,
attacks: none (Byzantines behave honestly), gaussian (N(0,1) noise), signflip (negated own knowledge),
labelflip (own knowledge with class order reversed), alie (coordinate-wise mean - 1.0*std of honest messages).
Reference points: average local accuracy of honest nodes before consensus ~ 0.684; accuracy of the
honest-only average (an oracle that knows who is Byzantine) ~ 0.860.

Existing rules (hand-designed, hyper-parameters already tuned on these scenarios)
Mean: score=0.2005 (mean=0.393, worst=0.008; none=0.818, gaussian=0.249, signflip=0.019, labelflip=0.017, alie=0.862)
```python
def aggregate(x_self, X_nb, t, T, state):
    return (x_self + X_nb.sum(0)) / (len(X_nb) + 1)
```
CW-Median: score=0.7831 (mean=0.831, worst=0.736; none=0.840, gaussian=0.848, signflip=0.826, labelflip=0.805, alie=0.834)
```python
def aggregate(x_self, X_nb, t, T, state):
    return np.median(np.concatenate([x_self[None], X_nb]), 0)
```
ClippedGossip: score=0.8193 (mean=0.847, worst=0.792; none=0.847, gaussian=0.853, signflip=0.833, labelflip=0.848, alie=0.853)
```python
def aggregate(x_self, X_nb, t, T, state):
    # self-centred clipping (He, Karimireddy & Jaggi 2022)
    k = len(X_nb)
    D = X_nb - x_self
    nrm = np.linalg.norm(D.reshape(k, -1), axis=1)
    tau = 0.5 * np.quantile(nrm, 0.25) + 1e-12
    scale = np.minimum(1.0, tau / np.maximum(nrm, 1e-12))
    return x_self + (scale[:, None, None] * D).sum(0) / (k + 1)
```
BALANCE: score=0.8215 (mean=0.854, worst=0.789; none=0.827, gaussian=0.861, signflip=0.861, labelflip=0.862, alie=0.862)
```python
def aggregate(x_self, X_nb, t, T, state):
    # distance-threshold acceptance (Fang et al., CCS 2024)
    nrm = np.linalg.norm((X_nb - x_self).reshape(len(X_nb), -1), axis=1)
    thr = 1.5 * np.exp(-1.0 * t / T) * np.linalg.norm(x_self)
    acc = X_nb[nrm <= thr]
    if len(acc) == 0:
        return x_self
    return 0.5 * x_self + 0.5 * acc.mean(0)
```

Best of your 4 previous candidates (feedback from the evaluation scenarios)

A1.4_ReputationClippedGossip (round 1): score=0.8364 (mean=0.856, worst=0.816; none=0.848, gaussian=0.859, signflip=0.858, labelflip=0.860, alie=0.857)
```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0, neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    k = len(Y)
    D = Y - xs[None]
    nrm = np.linalg.norm(D.reshape(k, -1), axis=1)
    tau = float(np.quantile(nrm, 0.25)) + 1e-12
    scale = np.minimum(1.0, tau / np.maximum(nrm, 1e-12))
    r = state.get('rep')
    if r is None or len(r) != k:
        r = scale.copy()
    else:
        r = 0.6 * r + 0.4 * scale
    state['rep'] = r
    w = r * scale
    out = xs + (w[:, None, None] * D).sum(0) / (1.0 + r.sum())
    out = np.clip(out, 0.0, None) + 1e-12
    return out / out.sum(-1, keepdims=True)

```

A1.2_AgreementTrust (round 1): score=0.8159 (mean=0.849, worst=0.782; none=0.821, gaussian=0.856, signflip=0.856, labelflip=0.856, alie=0.857)
```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0, neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    k = len(Y)
    C = xs.shape[-1]
    Z = np.concatenate([xs[None], Y], 0)
    med = np.median(Z, 0)
    ref = (0.5 * xs + 0.5 * med).argmax(-1)
    agree = (Y.argmax(-1) == ref[None]).mean(1)
    prev = state.get('trust')
    if prev is None or len(prev) != k:
        trust = agree
    else:
        trust = np.minimum(agree, 0.5 * prev + 0.5 * agree)
    state['trust'] = trust
    amax = float(trust.max())
    lo = max(2.0 / C, 0.5 * amax)
    hi = max(lo + 0.05, 0.85 * amax)
    w = np.clip((trust - lo) / (hi - lo), 0.0, 1.0)
    out = (xs + (w[:, None, None] * Y).sum(0)) / (1.0 + w.sum())
    return out

```

A1.1_HellingerGeoMedian (round 1): score=0.8140 (mean=0.847, worst=0.781; none=0.843, gaussian=0.854, signflip=0.850, labelflip=0.836, alie=0.853)
```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0, neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)
```
```

````
    xs = san(x_self)
    Y = san(X_nb)
    P = np.sqrt(np.concatenate([xs[None], Y], 0))
    n = P.shape[0]
    F = P.reshape(n, -1)
    z = np.median(F, 0)
    for _ in range(30):
        d = np.linalg.norm(F - z, axis=1)
        w = 1.0 / np.maximum(d, 1e-8)
        z_new = (w[:, None] * F).sum(0) / w.sum()
        if np.linalg.norm(z_new - z) < 1e-7:
            z = z_new
            break
        z = z_new
    ps = F[0]
    D = z - ps
    nd = np.linalg.norm(D)
    dn = np.linalg.norm(F[1:] - ps, axis=1)
    tau = float(np.median(dn)) + 1e-12
    step = D * min(1.0, tau / max(nd, 1e-12))
    out = (ps + step).reshape(xs.shape) ** 2
    out = out + 1e-12
    return out / out.sum(-1, keepdims=True)

```

### A1.3_PerSampleTrimmedHellinger (round 1): score=0.7647 (mean=0.825, worst
      =0.704; none=0.826, gaussian=0.837, signflip=0.795, labelflip=0.835, alie
      =0.834)
```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0,
      neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    A = np.sqrt(np.concatenate([xs[None], Y], 0))       # (n, m, C)
    n = A.shape[0]
    med = np.median(A, 0)
    ref = 0.5 * med + 0.5 * A[0]
    d = np.linalg.norm(A - ref[None], axis=2)           # (n, m)
    keep = max(1, (n + 1) // 2)
    idx = np.argsort(d, axis=0, kind='stable')[:keep]   # (keep, m)
    sel = np.take_along_axis(A, idx[:, :, None], axis=0)
    z = sel.mean(0)
    out = z ** 2 + 1e-12
    return out / out.sum(-1, keepdims=True)

```

## Results of your most recent round

- A1.1_HellingerGeoMedian: score=0.8140 (mean=0.847, worst=0.781; none=0.843,
      gaussian=0.854, signflip=0.850, labelflip=0.836, alie=0.853)

- A1.2_AgreementTrust: score=0.8159 (mean=0.849, worst=0.782; none=0.821,
      gaussian=0.856, signflip=0.856, labelflip=0.856, alie=0.857)

- A1.3_PerSampleTrimmedHellinger: score=0.7647 (mean=0.825, worst=0.704; none
      =0.826, gaussian=0.837, signflip=0.795, labelflip=0.835, alie=0.834)

- A1.4_ReputationClippedGossip: score=0.8364 (mean=0.856, worst=0.816; none
      =0.848, gaussian=0.859, signflip=0.858, labelflip=0.860, alie=0.857)

Analyse what worked and what failed, then improve: refine the strongest ideas,
      combine complementary mechanisms, and include at least one genuinely new
      idea.


## What to return
Propose 4 *diverse* candidate rules. For each: a short heading `### Candidate <
      name>`, 1-3 sentences on the idea,
then ONE ```python code block defining `aggregate`. Return nothing else. Do not
      use any tools.
````

Listing 4: Run A, round 1 response (verbatim), containing the selected program A1.4.

````
### Candidate HellingerGeoMedian

Project every message back onto the probability simplex (non-finite -> 0, clip,
      renormalise; all-zero rows -> uniform), embed in Hellinger space (square-
      root probabilities, where all honest messages lie on a bounded sphere),
      and compute the geometric median of self + neighbours with Weiszfeld
      iterations. The node then takes a self-centred step toward that median,
      capped at the median neighbour distance, and maps back by squaring.

```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0,
      neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    P = np.sqrt(np.concatenate([xs[None], Y], 0))
    n = P.shape[0]
    F = P.reshape(n, -1)
    z = np.median(F, 0)
    for _ in range(30):
        d = np.linalg.norm(F - z, axis=1)
        w = 1.0 / np.maximum(d, 1e-8)
        z_new = (w[:, None] * F).sum(0) / w.sum()
        if np.linalg.norm(z_new - z) < 1e-7:
            z = z_new
            break
        z = z_new
    ps = F[0]
    D = z - ps
    nd = np.linalg.norm(D)
    dn = np.linalg.norm(F[1:] - ps, axis=1)
    tau = float(np.median(dn)) + 1e-12
    step = D * min(1.0, tau / max(nd, 1e-12))
    out = (ps + step).reshape(xs.shape) ** 2
    out = out + 1e-12
    return out / out.sum(-1, keepdims=True)
```

### Candidate AgreementTrust
Each neighbour gets a trust score equal to the fraction of samples where its
      argmax matches a reference built half from the node's own state and half
      from the coordinate-wise median. The reference leans on the node's own
      state, so a colluding local majority cannot capture it, and the median
      half helps nodes with noisy labels. Trust is kept in memory so it drops
      fast and recovers slowly, and weights are set relative to the best-
      agreeing neighbour, with a floor above chance level so near-random or
      permuted messages get zero weight.

```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0,
      neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    k = len(Y)
    C = xs.shape[-1]
    Z = np.concatenate([xs[None], Y], 0)
    med = np.median(Z, 0)
    ref = (0.5 * xs + 0.5 * med).argmax(-1)
    agree = (Y.argmax(-1) == ref[None]).mean(1)
    prev = state.get('trust')
    if prev is None or len(prev) != k:
        trust = agree
    else:
        trust = np.minimum(agree, 0.5 * prev + 0.5 * agree)
    state['trust'] = trust
    amax = float(trust.max())
    lo = max(2.0 / C, 0.5 * amax)
    hi = max(lo + 0.05, 0.85 * amax)
    w = np.clip((trust - lo) / (hi - lo), 0.0, 1.0)
    out = (xs + (w[:, None, None] * Y).sum(0)) / (1.0 + w.sum())
    return out
```

### Candidate PerSampleTrimmedHellinger
````
````

````
Robust selection is done separately for each public sample, not per message. For
       each sample, after projecting onto the simplex and embedding in
       Hellinger space, the rule keeps the closest half of {self, neighbours} to
       a reference that is half own state and half coordinate-wise median, then
       averages them. An attacker therefore has to win sample by sample, and
       its influence on any one row is bounded.

```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0,
       neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    A = np.sqrt(np.concatenate([xs[None], Y], 0))       # (n, m, C)
    n = A.shape[0]
    med = np.median(A, 0)
    ref = 0.5 * med + 0.5 * A[0]
    d = np.linalg.norm(A - ref[None], axis=2)           # (n, m)
    keep = max(1, (n + 1) // 2)
    idx = np.argsort(d, axis=0, kind='stable')[:keep]   # (keep, m)
    sel = np.take_along_axis(A, idx[:, :, None], axis=0)
    z = sel.mean(0)
    out = z ** 2 + 1e-12
    return out / out.sum(-1, keepdims=True)
```

### Candidate ReputationClippedGossip
This is self-centred clipped gossip run on simplex-projected messages, plus a
       per-neighbour reputation kept in memory. The reputation is an exponential
       moving average of how much each neighbour's message had to be clipped,
       so neighbours that are far away again and again lose influence. The
       clipping radius adapts to a lower quantile of neighbour distances, and
       normalising by total reputation keeps any one Byzantine neighbour's
       displacement within that radius.

```python
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0,
       neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    k = len(Y)
    D = Y - xs[None]
    nrm = np.linalg.norm(D.reshape(k, -1), axis=1)
    tau = float(np.quantile(nrm, 0.25)) + 1e-12
    scale = np.minimum(1.0, tau / np.maximum(nrm, 1e-12))
    r = state.get('rep')
    if r is None or len(r) != k:
        r = scale.copy()
    else:
        r = 0.6 * r + 0.4 * scale
    state['rep'] = r
    w = r * scale
    out = xs + (w[:, None, None] * D).sum(0) / (1.0 + r.sum())
    out = np.clip(out, 0.0, None) + 1e-12
    return out / out.sum(-1, keepdims=True)
```
````

Listing 5: Selected program of run A (A1.4_ReputationClippedGossip.py).

```
def aggregate(x_self, X_nb, t, T, state):
    def san(X):
        X = np.nan_to_num(np.asarray(X, dtype=float), nan=0.0, posinf=0.0,
       neginf=0.0)
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        C = X.shape[-1]
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    xs = san(x_self)
    Y = san(X_nb)
    k = len(Y)
    D = Y - xs[None]
    nrm = np.linalg.norm(D.reshape(k, -1), axis=1)
    tau = float(np.quantile(nrm, 0.25)) + 1e-12
    scale = np.minimum(1.0, tau / np.maximum(nrm, 1e-12))
    r = state.get('rep')
    if r is None or len(r) != k:
        r = scale.copy()
    else:
        r = 0.6 * r + 0.4 * scale
    state['rep'] = r
    w = r * scale
    out = xs + (w[:, None, None] * D).sum(0) / (1.0 + r.sum())
    out = np.clip(out, 0.0, None) + 1e-12
    return out / out.sum(-1, keepdims=True)
```

Listing 6: Selected program of run B (B2.3_B2_3_TwoLevelRobustMean.py).

```
def aggregate(x_self, X_nb, t, T, state):
    C = x_self.shape[-1]

    def clean(X):
        X = np.nan_to_num(np.asarray(X, dtype=np.float64), nan=0.0, posinf=0.0,
       neginf=0.0)
        return np.clip(X, -1e6, 1e6)

    def proj(X):
        X = np.clip(X, 0.0, None)
        s = X.sum(-1, keepdims=True)
        return np.where(s > 1e-12, X / np.maximum(s, 1e-12), 1.0 / C)

    R = clean(X_nb)
    k = R.shape[0]
    P = proj(R)
    xs = proj(clean(x_self))
    viol = np.abs(R - P).sum(-1).mean(1)
    Z = np.concatenate([xs[None], P], 0)
    med = np.median(Z, 0)
    drow = np.abs(Z - med[None]).sum(-1)              # (n, m)
    dist = drow.mean(1)
    d_ref = float(np.median(dist)) + 1e-3
    w_dist = 1.0 / (1.0 + (dist / (2.0 * d_ref)) ** 4)
    lab = Z.argmax(-1)
    lab_c = med.argmax(-1)
    agree = 0.5 * ((lab == lab_c[None]).mean(1) + (lab == lab[0][None]).mean(1))
    w_agree = np.clip((agree - 2.5 / C) / (2.5 / C), 0.0, 1.0)
    w_new = (w_dist * w_agree)[1:] * np.exp(-viol / 0.05)
    prev = state.get('trust')
    if prev is None or len(prev) != k:
        trust = w_new
    else:
        trust = np.where(w_new < prev, w_new, 0.7 * prev + 0.3 * w_new)
    state['trust'] = trust
    tr = np.concatenate([[1.0], trust])
    s = np.median(drow, 0) + 0.05                      # (m,)
    u = 1.0 / (1.0 + (drow / (3.0 * s[None])) ** 2)
    W = tr[:, None] * u
    W[0] = np.maximum(W[0], 0.5)
    return (W[:, :, None] * Z).sum(0) / W.sum(0)[:, None]
```

Listing 7: Selected program of run C (C1.3_HellingerGM.py).

```
def aggregate(x_self, X_nb, t, T, state):
    def san(A):
        A = np.nan_to_num(np.asarray(A, dtype=float), nan=0.0, posinf=0.0,
       neginf=0.0)
        A = np.clip(A, 0.0, 1e6)
        s = A.sum(-1, keepdims=True)
        C = A.shape[-1]
        return np.where(s > 1e-12, A / np.maximum(s, 1e-12), 1.0 / C)

    x = san(x_self)
    X = san(X_nb)
    k = X.shape[0]
    S = np.sqrt(np.concatenate([x[None], X], 0))     # (n, m, C)
    wts = np.ones(k + 1)
    wts[0] = 1.0 + 0.25 * k
    y = np.median(S, 0)
    for _ in range(10):
        d = np.linalg.norm(S - y[None], axis=2)      # (n, m)
        inv = wts[:, None] / np.maximum(d, 1e-6)
        y = (inv[..., None] * S).sum(0) / inv.sum(0)[..., None]
    return san(y ** 2)
```

Listing 8: Selected program of run H (H5.1_H5_1_PersistentBandClippedMix.py).

```
def aggregate(x_self, X_nb, t, T, state):
    k = len(X_nb)
    if k == 0:
        return x_self
    P = np.concatenate([x_self[None], X_nb]).reshape(k + 1, -1).astype(float)
    P = np.nan_to_num(P, nan=0.0, posinf=0.0, neginf=0.0)
    ref = np.median(P, 0)
    d = np.linalg.norm(P - ref, axis=1)
    r = d / (np.median(d) + 1e-12)
    s_prev = state.get('s')
    if s_prev is None or len(s_prev) != k:
        s_prev = np.zeros(k)
    s = 0.6 * np.asarray(s_prev, dtype=float) + 0.4 * r[1:]
    state['s'] = s
    thr = 1.0 + math.exp(-2.0 * t / T)
    band = np.clip((thr - r[1:]) / (0.5 * thr + 1e-12), 0.0, 1.0)
    trust = band * np.exp(-np.maximum(s - 1.0, 0.0))
    x0 = P[0]
    D = P[1:] - x0
    nrm = np.linalg.norm(D, axis=1)
    tau = np.median(nrm) + 1e-12
    scale = np.minimum(1.0, tau / np.maximum(nrm, 1e-12))
    z = x0 + (trust * scale)[:, None].__mul__(D).sum(0) / (1.0 + trust.sum())
    z = np.nan_to_num(z, nan=0.0, posinf=0.0, neginf=0.0)
    return z.reshape(x_self.shape)
```

Listing 9: ARCO implementation used in all experiments.

```
"""ARCO: Anchored Robust Consensus Optimization.

Honest nodes jointly minimise
    F(X) = sum_i  alpha/2 ||x_i - y_i||^2  +  lambda/2 sum_i sum_{j in N_i}
       sum_s rho_delta(||x_i^s - x_j^s||)
over row-stochastic matrices x_i (one probability row per public sample s), by
       projected
(preconditioned) gradient descent.  One iteration = one message per neighbour.

    x_i <- Proj_simplex( x_i + gamma_i [ sum_j psi_delta(m_j - x_i) - kappa (x_i
       - y_i) ] )

psi_delta = gradient of rho_delta, applied row-wise:
    huber : psi(z) = z * min(1, delta/||z||)        (convex,   |psi| <= delta)
    log   : psi(z) = z * min(1, delta/||z||)^2      (redescending, weakly convex
       , |psi| <= delta)
kappa = alpha/lambda is the anchor strength; kappa = 0, huber, gamma=1/(k+1) is
       row-wise ClippedGossip.
"""
import numpy as np


def proj_simplex_rows(V):
    """Euclidean projection of every row (last axis) onto the probability
       simplex."""
    shp = V.shape
    V = V.reshape(-1, shp[-1])
    U = -np.sort(-V, axis=1)
    css = np.cumsum(U, axis=1) - 1.0
    idx = np.arange(1, V.shape[1] + 1)
    cond = U - css / idx > 0
    rho = cond.shape[1] - 1 - np.argmax(cond[:, ::-1], axis=1)
    theta = css[np.arange(len(V)), rho] / (rho + 1)
    return np.maximum(V - theta[:, None], 0.0).reshape(shp)


def psi(D, delta, loss):
    r = np.sqrt((D ** 2).sum(-1, keepdims=True))  # (k, m, 1) row distances
    w = np.minimum(1.0, delta / np.maximum(r, 1e-12))
    if loss == "log":
        w = w * w
    elif loss == "l2":
        w = np.ones_like(w)
    return w * D


def make_arco(kappa=0.1, delta=1.0, loss="log", solver="mm", group="node",
       adaptive=True, q=0.25, project=True, center="self"):
    """solver: 'mm'  = proximal majorize-minimize step (exact minimiser of the
       local quadratic majoriser)
               'grad'= projected gradient step with step 1/(k+1+kappa)
       group : 'node' = robust loss on whole-message distance, 'row' = per
       public sample
       adaptive: delta_t = delta * quantile_q(neighbour distances) (scale-free);
        else delta is absolute."""
    def aggregate(x_self, X_nb, t, T, state):
        if "y" not in state:
            state["y"] = proj_simplex_rows(np.nan_to_num(x_self)) if project
       else x_self.copy()
        x, y = x_self, state["y"]
        k = len(X_nb)
        M = np.nan_to_num(np.asarray(X_nb, dtype=float), nan=0.0, posinf=1e6,
       neginf=-1e6)
        if center == "median":  # high-breakdown reference point for measuring
       distances
            c = np.median(np.concatenate([x[None], M]), 0)
        else:
            c = x
        D = M - c
        if group == "node":
            r = np.sqrt((D ** 2).reshape(k, -1).sum(1)).reshape(k, 1, 1)
        else:
            r = np.sqrt((D ** 2).sum(-1, keepdims=True))
        dl = delta * np.quantile(r, q, axis=0, keepdims=True) if adaptive else
       delta
        w = np.minimum(1.0, dl / np.maximum(r, 1e-12))
        if loss == "log":
            w = w * w
        elif loss == "l2":
            w = np.ones_like(w)
        if solver == "mm":
            x = (x + kappa * y + (w * M).sum(0)) / (1.0 + kappa + w.sum(0))
        else:
            x = x + ((w * (M - x)).sum(0) - kappa * (x - y)) / (k + 1.0 + kappa)
        return proj_simplex_rows(x) if project else x
    return aggregate
```

Listing 10: The three frozen ARCO configurations.

```
"""ARCO configurations frozen from the design-side frontier (results/
      frontier_design.json)."""
from arco import make_arco
CONFIGS = {
    "ARCO-G": dict(loss="log", center="self", q=0.5, delta=0.5, kappa=0.1),    #
       max design-val score
    "ARCO-B": dict(loss="log", center="self", q=0.25, delta=2.0, kappa=0.1),   #
       max mean(val, drift40)
    "ARCO-H": dict(loss="huber", center="self", q=0.25, delta=1.0, kappa=0.1), #
       convex member (certified)
}
def get(name):
    return make_arco(**CONFIGS[name])
```

L DEVIATIONS FROM THE PRE-REGISTRATION

(1) The adaptive attack initially had a bug: its reference input had a different size from the attacked input for stateful rules. It was fixed, and the adaptive rows of *all* rules were recomputed.
(2) The drift attack and the ARCO family were introduced after the pre-registration, in response to the optimization analysis. ARCO configurations were selected on design-side data only. Earlier exploratory screens of ARCO variants touched held-out data.
(3) The Web-text suite (20 Newsgroups, 72 scenarios) and its end-to-end runs were added for this version, after all other results were final. No rule was re-tuned or re-selected after adding it.

M COMPUTE

All experiments ran on a single laptop (Apple M2, 8 GB unified memory) using the Metal GPU backend for training and CPU multi-processing for consensus scenarios. A CIFAR-10 end-to-end run (20 nodes, 20 rounds) takes about 5 minutes, and a 20 Newsgroups run about 40 seconds. The cost of the held-out suites is dominated by the adaptive attack, which queries the rule 25 times per message.